O/S overview

Overview

- 6.828 goals:
  - Understand operating systems in detail by designing and implementing a small O/S
  - Hands-on experience with building systems ("Applying 6.033")
- What do applications want from an O/S?
  - Abstract the hardware for convenience and portability
  - Multiplex the hardware among multiple applications
  - Isolate applications to contain bugs
  - Allow sharing among applications
- What is an OS?
  - e.g. OSX, Windows, Linux
  - the small view: a h/w management library
  - the big view: physical machine -> abstract one w/ better properties
- Organization: layered picture
    - h/w: CPU, mem, disk
    - kernel: [various services]
    - user: applications, e.g. vi and gcc
  - we care a lot about the interfaces and internel kernel structure
- what services does an O/S kernel typically provide?
  - processes
  - memory
  - file contents
  - directories and file names
  - security
  - many others: users, IPC, network, time, terminals
- What does an O/S abstraction look like?
  - Applications only see them via system calls
  - Examples, from UNIX / Linux:
  - `fd = open("out", 1);`
  - `write(fd, "hello\n", 6);`
  - `pid = fork();`
- Why is O/S design/implementation hard/interesting?
  - the environment is unforgiving: weird h/w, no debugger
  - it must be efficient (thus low-level?)
  - ...but abstract/portable (thus high-level?)
  - powerful (thus many features?)
  - ...but simple (thus a few composable building blocks?)
  - features interact: `fd = open(); ...; fork();`
  - behaviors interact: CPU priority vs memory allocator.
  - open problems: security, multi-core
- You'll be glad you learned about operating systems if you...
  - want to work on the above problems
  - care about what's going on under the hood
  - have to build high-performance systems
  - need to diagnose bugs or security problems

Class structure

- http://pdos.lcs.mit.edu/6.828
- Lectures
  - basic O/S ideas
  - extended inspection of xv6, a traditional O/S
  - several more recent topics
  - in-class programming
- Lab: JOS, a small O/S for x86 in an exokernel style
  - you build it, 5 labs, final project of your choice
  - kernel interface: expose hardware, but protect -- no abstractions!
  - unprivileged library: fork, exec, pipe, ...
  - applications: file system, shell, ..
  - development environment: gcc, qemu
  - lab 1 is out
- Code review
- One quiz: in class
- No final, but project and project conferences

Case study: the shell (simplified)

- 6.828 is largely about design and implementation of system call interface. let's start by looking at how programs use that interface. example: the Unix shell.
- the shell is the Unix command UI
- the shell is also a programming/scripting language
- typically handles login session, runs other processes
- look at some simple examples of shell operations, how they use different O/S abstractions, and how those abstractions fit together. See The UNIX Time-Sharing System' by D. M. Ritchie and K. Thompson if you are unfamiliar with the shell.
- Basic structure: see sh.c
- Basic organization: parsing and executing commands (e.g., ls, ls | wc, ls > out)
  Shell implemented using system calls (e.g., read, write, fork, exec, wait) conventions: -1 return value signals
- error, error code stored in `errno`, `perror` prints out a descriptive error message based on `errno`.
- Many systems calls are encapsulated in libc calls (e.g., fgets vs read)
  what does fork() do?
  copies user memory
  copies process kernel state (e.g. user id)
  child gets a different PID
  child state contains parent PID
- returns twice, with different values
  what does wait() do?
  waits for any child to exit
- what if child exits before parent calls wait?
- What does parscmd() do?
  Example:

```
$ ls
```

-
- The parse functions form a simple recursive-descent parser for the sh scripting language
- Do exercise 1 of Shell Exercises for Lecture 1
  You need a variant of exec() (type "man 3 exec"):
  replaces memory of current process with instrs/data from file
  i.e. runs a file created by compiler/linker
- still the same process, keeps most state (e.g. user id)
- the fork/exec split looks wasteful, but it turns out to be useful.
  how does "ls" know which directory to look at?
  cwd in kernel-maintained process state, copied during fork

- how does it know what to do with its output?
- I/O: process has file descriptors, numbered starting from 0.
  index into table in process's kernel state
- system calls: open, read, write, close
- numbering conventions:
  - file descriptor 0 for input (e.g., keyboard). fgets(stdin) invokes:

    ```
    read (0, buf, bufsize)
    ```

  - file descriptor 1 for output (e.g., terminal). fprintf(stdout) invokes:

    ```
    write (1, "hello\n", strlen("hello\n"))
    ```

  - file descriptor 2 for error (e.g., terminal)
- so ls writes output to file descriptor 1
- on fork, child inherits open file descriptors from parent (show in process diagram).
- on exec, process retains file descriptors.
- This shell command sends ls's output to the file out:

  ```
  $ ls > out
  ```

- Q: how could our simple shell implement output redirection?
- A: Do exercise 2.
- Good illustration of why it's nice to have separate fork and exec.
- Many commands use 0/1 by default, so they work with redirection. For example, see xv6 cat.c.
- system call interface simple, just ints and char buffers. why not have open() return a pointer reference to a kernel file object?
- Linux has a nice representation of a process and its FDs, under /proc/PID/
  - maps: VA range, perms (p=private, s=shared), offset, dev, inode, pathname
  - fd: symlinks to files pointed to by each fd. (what's missing in this representation?)
  - try `exec 3>/tmp/xx ; ls -l /proc/$$/fd`
- One often wants to run a series of programs on some data:

  ```
  $ sort < in > out
  $ uniq out > out2
  $ wc out2
  $ rm out out2
  ```

  the shell supports this more concisely with "piping" of "filters":

  ```
  $ sort < in | uniq | wc
  ```

- A pipe is a one-way communication channel. Here is a simple example:

  ```
  int fds[2];
  char buf[512];
  int n;

  pipe(fds);
  write(fds[1], "hello", 5);
  n = read(fds[0], buf, sizeof(buf));
  // buf[] now contains 'h', 'e', 'l', 'l', 'o'
  ```

- file descriptors are inherited across `fork()`, so this also works:

  ```
  int fds[2];
  char buf[512];
  int n, pid;

  pipe(fds);
  pid = fork();
  if(pid > 0){
    write(fds[1], "hello", 5);
  } else {
    n = read(fds[0], buf, sizeof(buf));
    exit(0);
  }
  ```

- How does the shell implement pipelines (i.e., cmd 1 | cmd 2 |..)? We want to arrange that the output of cmd 1 is the input of cmd 2. The way to achieve this goal is to manipulate stdout and stdin.
- The shell creates processes for each command in the pipeline, hooks up their stdin and stdout, and waits for the last process of the pipeline to exit.
- Do exercise 3.
- Who waits for whom? (draw a tree of processes)
- Why close read-end and write-end? ensure that every process starts with 3 file descriptors, and that reading from the pipe returns end of file after the first command exits.
- You can run the shell, redirect its stdin/stdout, etc.
- I'll run this shell script with `sh < script`:

```
echo one
echo two
```

- Q: what will this shell command do?

```
$ sh < script > out
```

- the script itself didn't redirect the echo output, but it did inherit a fd 1 that was redirected to out.
- Q: I'll run the following directly; is it the same as above?

```
echo one > out
echo two > out
```

- Notes about the file descriptor design:
  - nice interaction with fork
  - FDs help make programs more general purpose: don't need special cases for files vs console vs pipe
  - shell pipelines only work for programs w/ common formats (lines of text)
- How do you create a background job?

```
$ sleep 2 &
```

- Q: How does the shell implement "&"?
- Q: What if a background process exits while sh waits for a foreground process?
- Do challenge exercises

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 Lecture Notes: x86 and PC architecture

Outline

- PC architecture
- x86 instruction set
- gcc calling conventions
- PC emulation

PC architecture

- A full PC has:
  - an x86 CPU with registers, execution unit, and memory management
  - CPU chip pins include address and data signals
  - memory
  - disk
  - keyboard
  - display
  - other resources: BIOS ROM, clock, ...
- We will start with the original 16-bit 8086 CPU (1978)
- CPU runs instructions:

```
for(;;){
        run next instruction
}
```

- Needs work space: registers
  - four 16-bit data registers: AX, BX, CX, DX
  - each in two 8-bit halves, e.g. AH and AL
  - very fast, very few
- More work space: memory
  - CPU sends out address on address lines (wires, one bit per wire)
  - Data comes back on data lines
  - *or* data is written to data lines
- Add address registers: pointers into memory
  - SP - stack pointer
  - BP - frame base pointer
  - SI - source index
  - DI - destination index
- Instructions are in memory too!
  - IP - instruction pointer (PC on PDP-11, everything else)
  - increment after running each instruction
  - can be modified by CALL, RET, JMP, conditional jumps
- Want conditional jumps
  - FLAGS - various condition codes
    - whether last arithmetic operation overflowed
    - ... was positive/negative
    - ... was [not] zero
    - ... carry/borrow on add/subtract
    - ... etc.
    - whether interrupts are enabled
    - direction of data copy instructions

- JP, JN, J[N]Z, J[N]C, J[N]O ...
- Still not interesting - need I/O to interact with outside world
  - Original PC architecture: use dedicated *I/O space*
    - Works same as memory accesses but set I/O signal
    - Only 1024 I/O addresses
    - Accessed with special instructions (IN, OUT)
    - Example: write a byte to line printer:

```
#define DATA_PORT    0x378
#define STATUS_PORT  0x379
#define   BUSY 0x80
#define CONTROL_PORT 0x37A
#define   STROBE 0x01
void
lpt_putc(int c)
{
  /* wait for printer to consume previous byte */
  while((inb(STATUS_PORT) & BUSY) == 0)
    ;

  /* put the byte on the parallel lines */
  outb(DATA_PORT, c);

  /* tell the printer to look at the data */
  outb(CONTROL_PORT, STROBE);
  outb(CONTROL_PORT, 0);
}
```

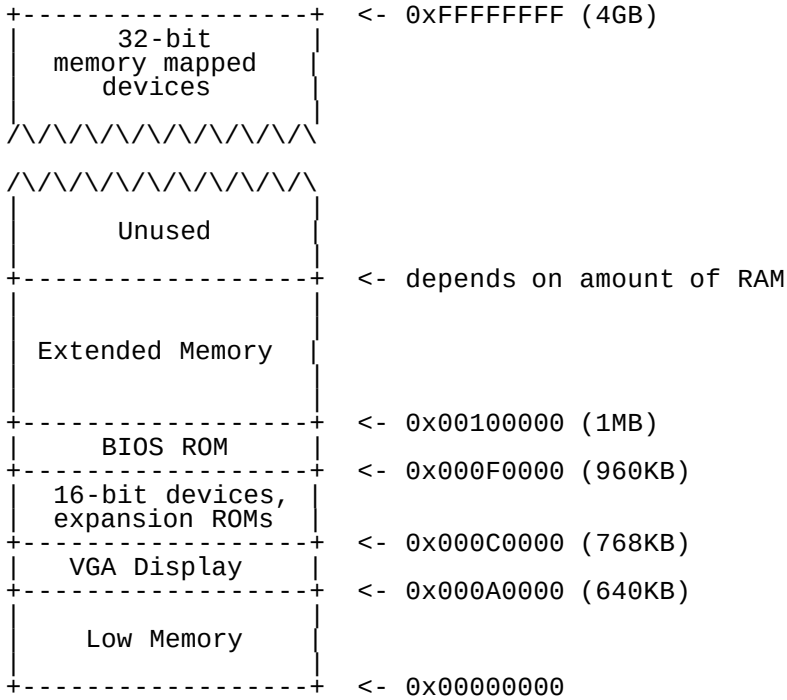
  - Memory-Mapped I/O
    - Use normal physical memory addresses
      - Gets around limited size of I/O address space
      - No need for special instructions
      - System controller routes to appropriate device
    - Works like ``magic" memory:
      - *Addressed* and *accessed* like memory, but ...
      - ... does not *behave* like memory!
      - Reads and writes can have ``side effects"
      - Read results can change due to external events
- What if we want to use more than 2^16 bytes of memory?
  - 8086 has 20-bit physical addresses, can have 1 Meg RAM
  - the extra four bits usually come from a 16-bit "segment register":
  - CS - code segment, for fetches via IP
  - SS - stack segment, for load/store via SP and BP
  - DS - data segment, for load/store via other registers
  - ES - another data segment, destination for string operations
  - virtual to physical translation: pa = va + seg*16
  - e.g. set CS = 4096 to execute starting at 65536
  - tricky: can't use the 16-bit address of a stack variable as a pointer
  - a *far pointer* includes full segment:offset (16 + 16 bits)
  - tricky: pointer arithmetic and array indexing across segment boundaries
- But 8086's 16-bit addresses and data were still painfully small
  - 80386 added support for 32-bit data and addresses (1985)
  - boots in 16-bit mode, boot.S switches to 32-bit mode
  - registers are 32 bits wide, called EAX rather than AX
  - operands and addresses that were 16-bit became 32-bit in 32-bit mode, e.g. ADD does 32-bit arithmetic
  - prefixes 0x66/0x67 toggle between 16-bit and 32-bit operands and addresses: in 32-bit mode, MOVW is expressed as 0x66 MOVW
  - the .code32 in boot.S tells assembler to generate 0x66 for e.g. MOVW
  - 80386 also changed segments and added paged memory...

- Example instruction encoding

```
b8 cd ab                16-bit CPU,  AX <- 0xabcd
b8 34 12 cd ab          32-bit CPU, EAX <- 0xabcd1234
66 b8 cd ab             32-bit CPU,  AX <- 0xabcd
```

x86 Physical Memory Map

- The physical address space mostly looks like ordinary RAM
- Except some low-memory addresses actually refer to other things
- Writes to VGA memory appear on the screen
- Reset or power-on jumps to ROM at 0xffffff0 (so must be ROM at top...)



x86 Instruction Set

- Intel syntax: `op dst, src` (Intel manuals!)
- AT&T (gcc/gas) syntax: `op src, dst` (labs, xv6)
  - uses b, w, l suffix on instructions to specify size of operands
- Operands are registers, constant, memory via register, memory via constant
- Examples:

AT&T syntax	"C"-ish equivalent	
movl %eax, %edx	edx = eax;	*register mode*
movl $0x123, %edx	edx = 0x123;	*immediate*
movl 0x123, %edx	edx = *(int32_t*)0x123;	*direct*
movl (%ebx), %edx	edx = *(int32_t*)ebx;	*indirect*
movl 4(%ebx), %edx	edx = *(int32_t*)(ebx+4);	*displaced*

- Instruction classes
  - data movement: MOV, PUSH, POP, ...
  - arithmetic: TEST, SHL, ADD, AND, ...
  - i/o: IN, OUT, ...
  - control: JMP, JZ, JNZ, CALL, RET

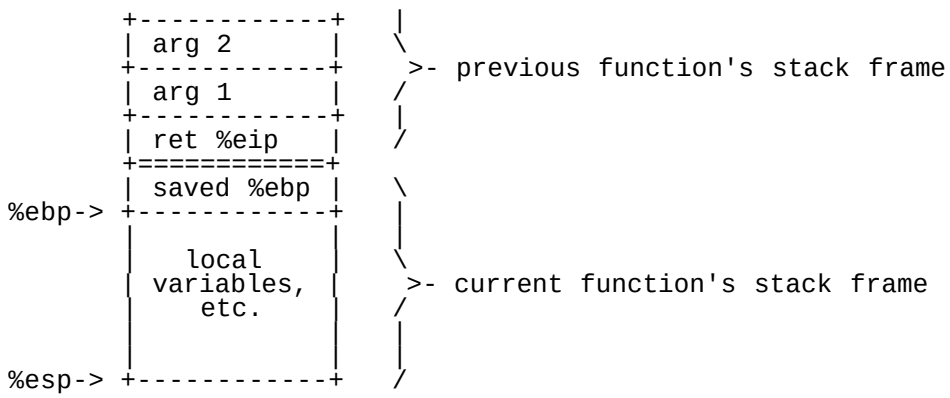
    - string: REP MOVSB, ...
    - system: IRET, INT
- Intel architecture manual Volume 2 is *the* reference

gcc x86 calling conventions

- x86 dictates that stack grows down:

Example instruction	What it does
pushl %eax	subl $4, %esp movl %eax, (%esp)
popl %eax	movl (%esp), %eax addl $4, %esp
call 0x12345	pushl %eip (*) movl $0x12345, %eip (*)
ret	popl %eip (*)

  (*) *Not real instructions*
- GCC dictates how the stack is used. Contract between caller and callee on x86:
    - at entry to a function (i.e. just after call):
        - %eip points at first instruction of function
        - %esp+4 points at first argument
        - %esp points at return address
    - after ret instruction:
        - %eip contains return address
        - %esp points at arguments pushed by caller
        - called function may have trashed arguments
        - %eax (and %edx, if return type is 64-bit) contains return value (or trash if function is `void`)
        - %eax, %edx (above), and %ecx may be trashed
        - %ebp, %ebx, %esi, %edi must contain contents from time of `call`
    - Terminology:
        - %eax, %ecx, %edx are "caller save" registers
        - %ebp, %ebx, %esi, %edi are "callee save" registers
- Functions can do anything that doesn't violate contract. By convention, GCC does more:
    - each function has a stack frame marked by %ebp, %esp



    - %esp can move to make stack frame bigger, smaller
    - %ebp points at saved %ebp from previous function, chain to walk stack
    - function prologue:

```
pushl %ebp
```

```
movl %esp, %ebp
```

or

```
enter $0, $0
```

enter usually not used: 4 bytes vs 3 for pushl+movl, not on hardware fast-path anymore

- function epilogue can easily find return EIP on stack:

```
movl %ebp, %esp
popl %ebp
```

or

```
leave
```

leave used often because it's 1 byte, vs 3 for movl+popl

- Big example:
  - C code

```
int main(void) { return f(8)+1; }
int f(int x) { return g(x); }
int g(int x) { return x+3; }
```

  - assembler

```
_main:
                        prologue
        pushl %ebp
        movl %esp, %ebp
                        body
        pushl $8
        call _f
        addl $1, %eax
                        epilogue
        movl %ebp, %esp
        popl %ebp
        ret
_f:
                        prologue
        pushl %ebp
        movl %esp, %ebp
                        body
        pushl 8(%esp)
        call _g
                        epilogue
        movl %ebp, %esp
        popl %ebp
        ret

_g:
                        prologue
        pushl %ebp
        movl %esp, %ebp
                        save %ebx
        pushl %ebx
                        body
        movl 8(%ebp), %ebx
        addl $3, %ebx
        movl %ebx, %eax
                        restore %ebx
        popl %ebx
                        epilogue
        movl %ebp, %esp
        popl %ebp
        ret
```

- Super-small _g:

```
_g:
        movl 4(%esp), %eax
        addl $3, %eax
        ret
```

- Shortest `_f`?
- Compiling, linking, loading:
  - *Preprocessor* takes C source code (ASCII text), expands #include etc, produces C source code
  - *Compiler* takes C source code (ASCII text), produces assembly language (also ASCII text)
  - *Assembler* takes assembly language (ASCII text), produces `.o` file (binary, machine-readable!)
  - *Linker* takes multiple '`.o`'s, produces a single *program image* (binary)
  - *Loader* loads the program image into memory at run-time and starts it executing

PC emulation

- The Bochs emulator works by
  - doing exactly what a real PC would do,
  - only implemented in software rather than hardware!
- Runs as a normal process in a "host" operating system (e.g., Linux)
- Uses normal process storage to hold emulated hardware state: e.g.,
  - Stores emulated CPU registers in global variables

```
int32_t regs[8];
#define REG_EAX 1;
#define REG_EBX 2;
#define REG_ECX 3;
...
int32_t eip;
int16_t segregs[4];
...
```

  - Stores emulated physical memory in Boch's memory

```
char mem[256*1024*1024];
```

- Execute instructions by simulating them in a loop:

```
for (;;) {
        read_instruction();
        switch (decode_instruction_opcode()) {
        case OPCODE_ADD:
                int src = decode_src_reg();
                int dst = decode_dst_reg();
                regs[dst] = regs[dst] + regs[src];
                break;
        case OPCODE_SUB:
                int src = decode_src_reg();
                int dst = decode_dst_reg();
                regs[dst] = regs[dst] - regs[src];
                break;
        ...
        }
        eip += instruction_length;
}
```

- Simulate PC's physical memory map by decoding emulated "physical" addresses just like a PC would:

```
#define KB              1024
#define MB              1024*1024

#define LOW_MEMORY      640*KB
#define EXT_MEMORY      10*MB

uint8_t low_mem[LOW_MEMORY];
uint8_t ext_mem[EXT_MEMORY];
```

```
uint8_t bios_rom[64*KB];

uint8_t read_byte(uint32_t phys_addr) {
        if (phys_addr < LOW_MEMORY)
                return low_mem[phys_addr];
        else if (phys_addr >= 960*KB && phys_addr < 1*MB)
                return rom_bios[phys_addr - 960*KB];
        else if (phys_addr >= 1*MB && phys_addr < 1*MB+EXT_MEMORY) {
                return ext_mem[phys_addr-1*MB];
        else ...
}

void write_byte(uint32_t phys_addr, uint8_t val) {
        if (phys_addr < LOW_MEMORY)
                low_mem[phys_addr] = val;
        else if (phys_addr >= 960*KB && phys_addr < 1*MB)
                ; /* ignore attempted write to ROM! */
        else if (phys_addr >= 1*MB && phys_addr < 1*MB+EXT_MEMORY) {
                ext_mem[phys_addr-1*MB] = val;
        else ...
}
```

- Simulate I/O devices, etc., by detecting accesses to "special" memory and I/O space and emulating the correct behavior: e.g.,
  - Reads/writes to emulated hard disk transformed into reads/writes of a file on the host system
  - Writes to emulated VGA display hardware transformed into drawing into an X window
  - Reads from emulated PC keyboard transformed into reads from X input event queue

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2012 Lecture 3: O/S Organization

plan:
  O/S organization
  processes
  isolation

topic: overall o/s design
  what should the main components be?
  what should the interfaces look like?

why have an o/s at all?
  why not just a library?
  then apps are free to use it, or not -- flexible
  some tiny O/Ss for embedded processors work this way

key requirement: support multiple activities
  multiplexing
  isolation
  interaction

helpful approach:
  abstract services rather than raw hardware
  file system, not raw disk
  TCP, not raw ethernet
  processes, not raw CPU/memory
  abstractions often ease multiplexing and interaction
    and more convenient and portable

note:
  i'm going to focus on mainstream designs (xv6, Linux, &c)
  for *every* aspect, someone has done it a different way!
  example: exokernel and VMM does *not* abstract anything!

xv6 has only a few abstractions / services
  processes (cpu, mem)
  I/O (file descriptors)
  file system

i'm going to focus on xv6 processes today
  a process is a running program
  it has its own memory, share of CPU, FDs, parent, children, &c
  it uses system calls to interact outside itself
    to get at kernel services
  xv6 basic design here very traditional (UNIX/Linux/&c)

xv6 user/kernel organization
  h/w, kernel, user
  kernel is a big program
    services: process, FS, net
    low-level: devices, VM
  all of kernel runs w/ full hardware privilege (very convenient)
  system calls switch between user and kernel

good: easy for sub-systems to cooperate (e.g. paging and file system)
bad: interactions => complex, bugs are easy, no isolation within o/s
called "monolithic"; traditional and successful
worth thinking about what *has* to be in the kernel
Q: could FS be a user-level library? why / why not?
note: you could have a small kernel, most functionality at user-level
microkernel, exokernel

isolation is the most constraining consideration!
isolation determines much of the basic design
it's much of the reason why we need the notion of process at all
isolation will come up again and again

what is isolation?
the process is the unit of isolation
prevent process X from wrecking or spying on process Y
memory, cpu, FDs, resource exhaustion
prevent a process from wrecking the operating system itself
i.e. from preventing kernel from enforcing isolation
in the face of bugs or malice
e.g. a bad process may try to trick the h/w or kernel

what are all the mechanisms that keep processes isolated?
user/kernel mode flag
address spaces
timeslicing
system call interface

the foundation of xv6's isolation: user/kernel mode flag
controls whether instructions can access privileged h/w
called CPL on the x86, bottom two bits of %cs
CPL=0 -- kernel mode -- privileged
CPL=3 -- user mode -- no privilege
x86 CPL protects everything relevant to isolation
writes to %cs (to defend CPL)
every memory read/write
I/O port accesses
control register accesses (eflags, %cs4, ...)
every serious microprocessor has something similar

user/kernel mode flag is not enough
protects only against direct attacks on the hardware
kernel must configure control regs, page tables, &c to protect other stuff
e.g. kernel memory

how to do a system call -- switching CPL
Q: would this be an OK design for user programs to make a system call:
set CPL=0
jmp sys_open
bad: user-specified instructions with CPL=0
Q: how about a combined instruction that sets CPL=0,
but *requires* an immediate jump to someplace in the kernel?
bad: user might jump somewhere awkward in the kernel
the x86 answer:

there are only a few permissible kernel entry points
INT instruction sets CPL=0 and jumps to an entry point
but user code can't otherwise modify CPL or jump anywhere else in kernel
system call return sets CPL=3 before returning to user code
also a combined instruction (can't separately set CPL and jmp)
but kernel is allowed to jump anywhere in user code

the result: well-defined notion of user vs kernel
either CPL=3 and executing user code
or CPL=0 and executing from entry point in kernel code
not:
CPL=0 and executing user
CPL=0 and executing anywhere in kernel the user pleases

Q: could one have process isolation WITHOUT h/w-supported kernel/user mode?
yes!
see Singularity O/S, later in semester
but h/w user/kernel mode is the most popular plan

how to isolate process memory?
idea: "address space"
give each process some memory it can access
for its code, variables, heap, stack
prevent it from accessing other memory (kernel or other processes)

how to create isolated address spaces?
xv6 uses x86 "paging hardware"
MMU translates (or "maps") every address issued by program
VA -> PA
instruction fetch, data load/store
for kernel and user
there's no way for any instruction to directly use a PA
MMU array w/ entry for each 4k range of "virtual" address space
refers to phy address for that "page"
this is the page table
o/s tells h/w to switch page table when switching process
why isolated?
each page table entry (PTE) has a bit saying if user-mode instructions can use
kernel only sets the bit for the memory in current process's address space
paging h/w used in many ways, not just isolation
e.g. copy-on-write fork(), see Lab 4
note: you don't need paging to isolate memory
type safety, JVM, Singularity
but paging is the most popular plan

how to isolate CPU?
prevent a process from hogging the CPU, e.g. buggy infinite loop
how to force uncooperative process to yield
h/w provides a periodic "clock interrupt"
forcefully suspends current process
jumps into kernel
which can switch to a different process
kernel must save/restore process state (registers)
totally transparent, even to cooperative processes

called "pre-emptive context switch"
note: traditional, but maybe not perfect; see exokernel paper

back to system calls
i've talked a lot about how o/s isolates processes
but need user/kernel to cooperate! user needs kernel services.
what should user/kernel interaction look like?
can't let user r/w kernel mem (well, you can, later...)
kernel can r/w user mem
  but don't want to do this too much!
so style of system call interface is pretty simple
  integers, strings (copying only), user-allocated buffers
  no objects, data structures, &c
  never any doubt about who owns memory
let's illustrate by tracing sys calls in xv6

on-screen:
xterm -fn 10x20
illustrate sh.c exercise
draw parent/child diagram
echo hi
echo hi > x
echo hi | wc

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2012 Lecture 4: Virtual Memory

plan:
  address spaces
  paging hardware
  xv6 VM code

today's problem:
  [user/kernel diagram]
  suppose the shell has a bug:
    sometimes it writes to a random memory address
  how can we keep it from wrecking the kernel?
    and from wrecking other processes?

we want isolated address spaces
  each process has its own memory
  it can read and write its own memory
  it cannot read or write anything else

xv6 uses x86's paging hardware to implement AS's
  ask questions! this material is important

paging provides a level of indirection for addressing
  CPU -> MMU -> RAM
     VA   PA
  s/w can only ld/st to virtual addresses, not physical
  kernel tells MMU how to map each virtual address to a physical address
    MMU essentially has a table, indexed by va, yielding pa
    called a "page table"
  MMU can restrict what virtual addresses user code can use

x86 maps 4-KB "pages"
  and aligned -- start on 4 KB boundaries
  thus page table index is top 20 bits of VA

what is in a page table entry (PTE)?
  see handout
  top 20 bits are top 20 bits of physical address
    "physical page number"
    MMU replaces top 20 of VA with PPN
  low 12 bits are flags
    Present, Writeable, &c

where is the page table stored?
  in RAM -- MMU loads (and stores) PTEs
  o/s can read/write PTEs

would it be reasonable for page table to just be an array of PTEs?
  how big is it?
  2^20 is a million
  32 bits per entry
  4 MB for a full page table -- pretty big on early machines
  would waste lots of memory for small programs!

you only need mappings for a few hundred pages
so the rest of the million entries would be there but not needed

x86 uses a "two-level page table" to save space
diagram
pages of PTEs in RAM
page directory (PD) in RAM
PDE also contains 20-bit PPN -- of a page of 1024 PTEs
1024 PDEs point to PTE pages
each PTE page has 1024 PTEs -- so 1024*1024 PTEs in total
PD entries can be invalid
those PTE pages need not exist
so a page table for a small address space can be small

how does the mmu know where the page table is located in RAM?
%cr3 holds phys address of PD
PD holds phys address of PTE pages
they can be anywhere in RAM -- need not be contiguous

how does x86 paging hardware translate a va?
need to find the right PTE
%cr3 points to PA of PD
top 10 bits index PD to get PA of PT
next 10 bits index PT to get PTE
PPN from PTE + low-12 from VA

flags in PTE
P, W, U
xv6 uses U to forbid user from using kernel memory

what if P bit not set? or store and W bit not set?
"page fault"
CPU saves registers, forces transfer to kernel
trap.c in xv6 source
kernel can just produce error, kill process
or kernel can install a PTE, resume the process
e.g. after loading the page of memory from disk

Q: why mapping rather than e.g. base/bound?
indirection allows paging h/w to solve many problems
e.g. avoids fragmentation
e.g. copy-on-write fork
e.g. lazy allocation (today's in-class exercise)
many more techniques

how does xv6 use the x86 paging hardware?

big picture of an xv6 address space -- one per process
[diagram]
0x00000000:0x80000000 -- user addresses below KERNBASE
0x80000000:0x80100000 -- map low 1MB devices (for kernel)
0x80100000:? -- kernel instructions/data
? :0x8E000000 -- 224 MB of DRAM mapped here
0xFE000000:0x00000000 -- more memory-mapped devices

where does xv6 map these regions, in phys mem?
  [diagram]
  note double-mapping of user pages

each process has its own address space
  and its own page table
  all processes have the same kernel (high memory) mappings
  kernel switches page tables (i.e. sets %cr3) when switching processes

Q: why this address space arrangement?
  user virtual addresses start at zero
    of course user va 0 maps to different pa for each process
  2GB for user heap to grow contiguously
    but needn't have contiguous phys mem -- no fragmentation problem
  both kernel and user mapped -- easy to switch for syscall, interrupt
  kernel mapped at same place for all processes
    eases switching between processes
  easy for kernel to r/w user memory
    using user addresses, e.g. sys call arguments
  easy for kernel to r/w physical memory
    pa x mapped at va x+0x80000000
    we'll see this soon while manipulating page tables

Q: what's the largest process this scheme can accomodate?

Q: could we increase that by increasing/decreasing 0x80000000?

To think about: does the kernel have to map all of phys mem
  into its virtual address space?

let's look at some xv6 virtual memory code
  virtual memory == address space / translation

a process calls sbrk(n) to ask for n more bytes of heap memory
  malloc() uses sbrk()
  each process has a size
    kernel adds new memory at process's end, increases size
  sbrk() allocates physical memory (RAM)
  maps it into the process's page table
  returns the starting address of the new memory

sys_sbrk() in sysproc.c

growproc() in proc.c
  proc->sz is the process's current size
  allocuvm() does most of the work
  switchuvm sets %cr3 with new page table
    also flushes some MMU caches so it will see new PTEs

allocuvm() in vm.c
  why if(newsz >= KERNBASE) ?
  why PGROUNDUP?
  arguments to mappages()...

mappages() in vm.c
  arguments are PD, va, size, pa, perm
  adds mappings from a range of va's to corresponding pa's
  rounds b/c some uses pass in non-page-aligned addresses
  for each page-aligned address in the range
    call walkpgdir to find address of PTE
      need the PTE's address (not just content) b/c we want to modify
    put the desired pa into the PTE
    mark PTE as valid w/ PTE_P

diagram of PD &c, as following steps build it

walkpgdir() in vm.c
  mimics how the paging h/w finds the PTE for an address
  refer to the handout
  PDX extracts top ten bits
  &pgdir[PDX(va)] is the address of the relevant PDE
  now *pde is the PDE
  if PTE_P
    the relevant page-table page already exists
    PTE_ADDR extracts the PPN from the PDE
    p2v() adds 0x80000000, since PTE holds physical address
  if not PTE_P
    alloc a page-table page
    fill in PDE with PPN -- thus v2p
  now the PTE we want is in the page-table page
    at offset PTX(va)
    which is 2nd 10 bits of va

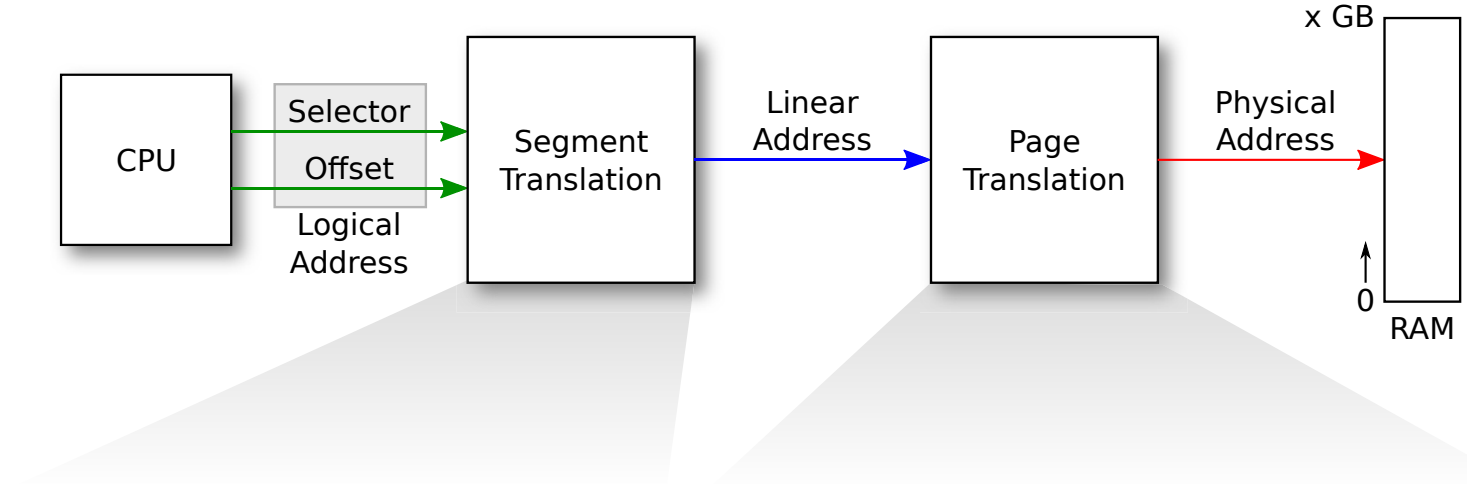
MIT OpenCourseWare
http://ocw.mit.edu

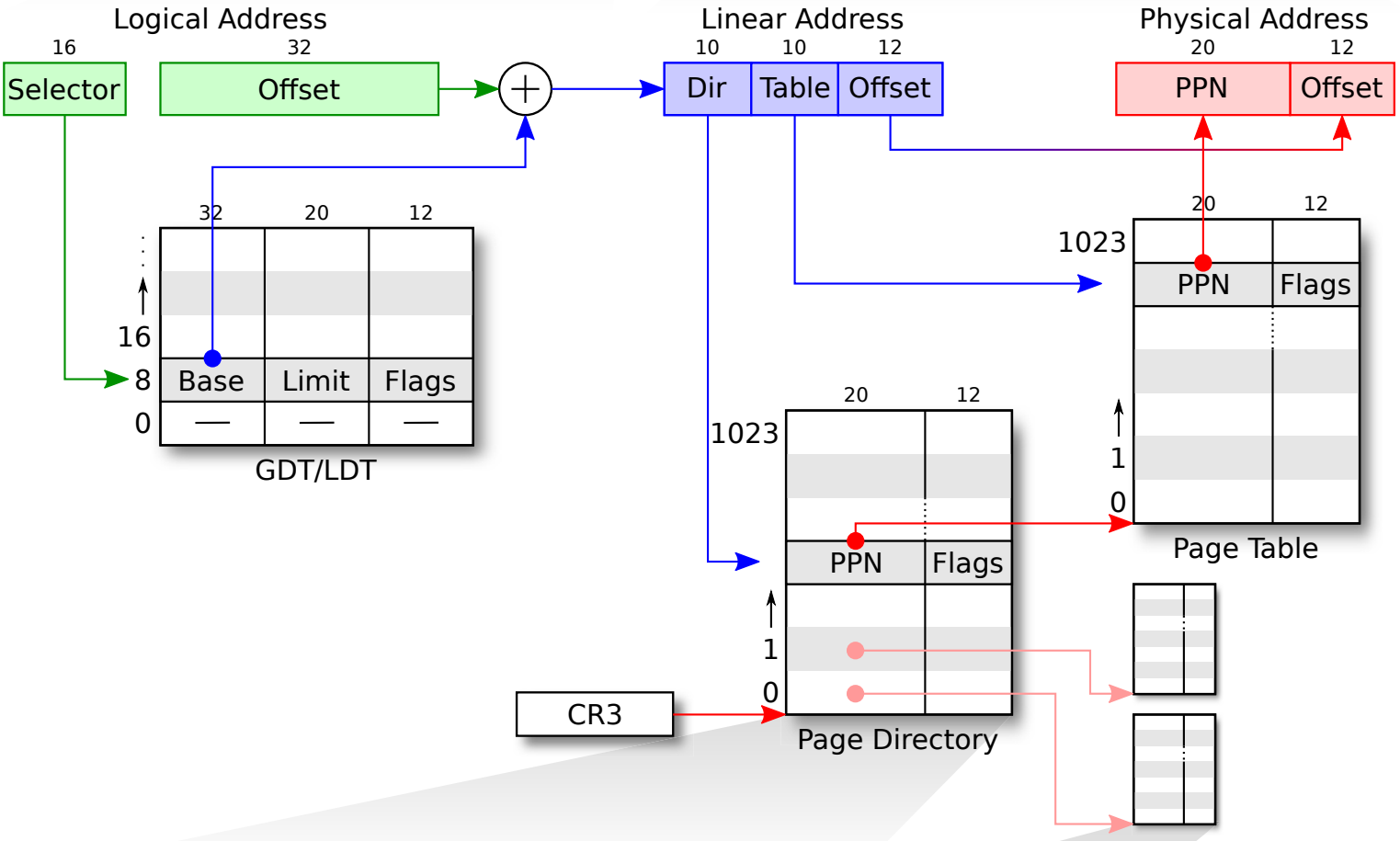
6.828 Operating System Engineering

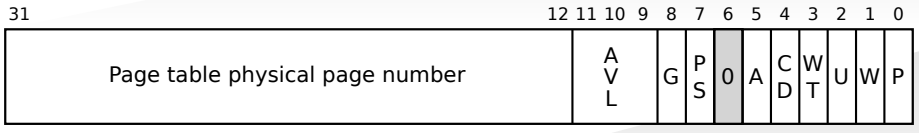
Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

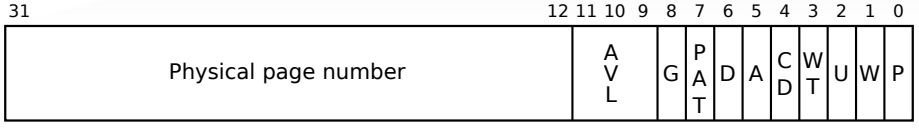
Protected-Mode Address Translation







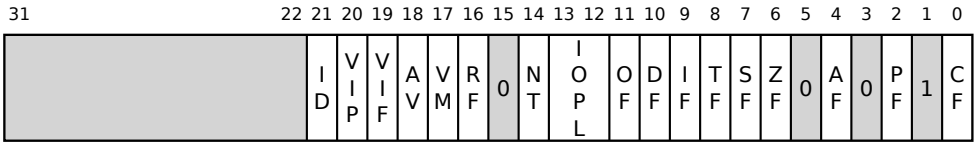
PDE



PTE

P	Present
W	Writable
U	User
WT	1=Write-through, 0=Write-back
CD	Cache disabled
A	Accessed
D	Dirty
PS	Page size (0=4KB, 1=4MB)
PAT	Page table attribute index
G	Global page
AVL	Available for system use

EFLAGS



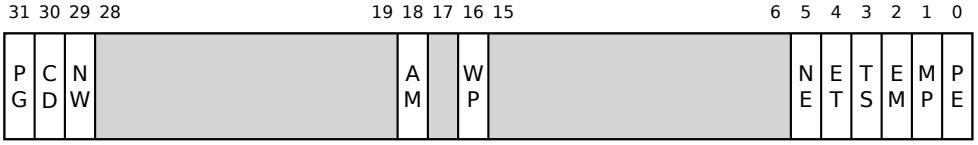
Status flags
- CF Carry flag
- PF Parity flag
- AF Auxiliary carry flag
- ZF Zero flag
- SF Sign flag
- OF Overflow flag

Control flags
- DF Direction flag

System flags
- TF Trap flag
- IF Interrupt enable flag
- IOPL I/O privilege level
- NT Nested task

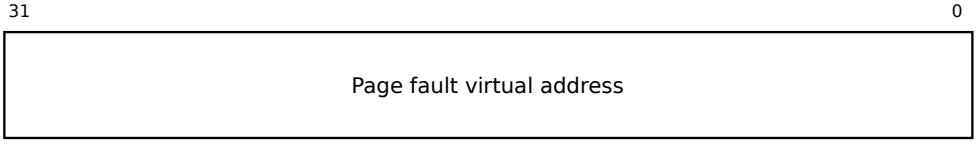
- RF Resume flag
- VM Virtual-8086 mode
- AC Alignment check
- VIF Virtual intr. flag
- VIP Virtual intr. pending
- ID ID flag

CR0

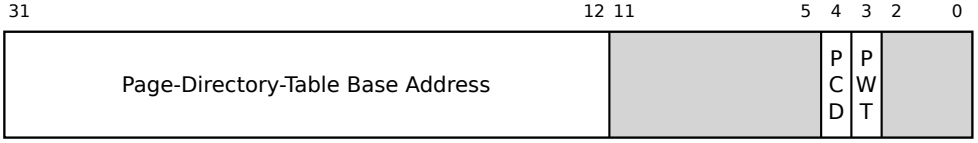


- PE Protection enabled
- MP Monitor coprocessor
- EM Emulation
- TS Task switched
- ET Extension type
- NE Numeric error
- WP Write protect
- AM Alignment mask
- NW Not write-through
- CD Cache disable
- PG Paging

CR2

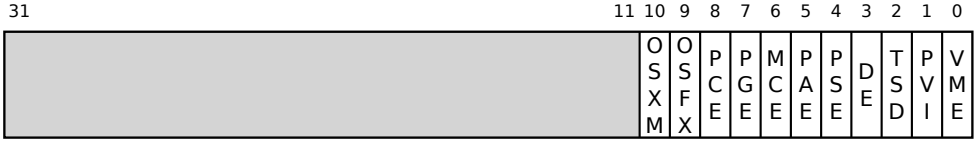


CR3



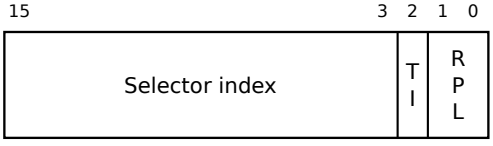
- PWT Page-level writes transparent
- PCT Page-level cache disable

CR4



- VME Virtual-8086 mode extensions
- PVI Protected-mode virtual interrupts
- TSD Time stamp disable
- DE Debugging extensions
- PSE Page size extensions
- PAE Physical-address extension
- MCE Machine check enable
- PGE Page-global enable
- PCE Performance counter enable
- OSFXSR OS FXSAVE/FXRSTOR support
- OSXMM-EXCPT OS unmasked exception support

CS/DS/ES
FS/GS/SS



- TI Table index (0=GDT, 1=LDT)
- RPL Requester privilege level

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2012 Lecture 5: Interrupts, exceptions

plan:
  entering the kernel (interrupts, system calls, &c)
  returning to user space

where can the system be executing?
  diagram: u/k, user stacks, kernel stacks

what are the transitions?
  u -> k: sys call / interrupt / exception
  k -> u: return from sys call / interrupt / exception
  k -> k: context switch
  k -> k: interrupt in kernel mode

interrupts, exceptions, and system calls all use the same mechanism!
we'll talk about context switch next week

Q: why per-process kernel stack?
   what would go wrong if syscall used a single global stack?

*** entering the kernel

why do we need to take special care for user -> kernel?
  we want to maintain isolation
  we want transparency (esp for device interrupts)

remember how x86 privilege levels work
  CPL in low 2 bits of CS
  CPL=0 -> kernel mode
  CPL=3 -> user mode

what has to happen in a system call?
  save user state for future resume
  set up for execution in kernel (stack, CPL=0)
  choose a place to execute in kernel
  get at system call arguments

let's look at what happens during a system call
  an sbrk() call in sh
  b *0xf48
  c -- to mov $0xc, %eax -- 0xc = 12 = SYS_sbrk
  stepi -- to int $0x40
  x/4x $esp -- return PC, sbrk argument, ...
  stepi -- to vector64, pushl $0x40

where are we? how did we get here?

the INT instruction jumps into the kernel

where does INT jump to?
  the $0x40 is a vector number
  a vector is an allowed entry point

x86 has 256 vectors, for different uses (devices, exceptions, &c)
kernel knows why interrupt occured by looking at vector #
vector is index of a descriptor in the "IDT"
IDTR register has IDT's base address
each IDT descriptor has seg selector, offset in segment
see handout
for xv6, seg selector always SEG_KCODE, offset is address of vector fn
IDT seg selector will be the code segment
so IDT seg selector determines CPL
print idt[0x40]

INT instruction steps (similar for interrupts and exceptions):
fetch vector's descriptor from IDT
if seg selector's PL < CPL:
it's a cross-ring interrupt
save ESP and SS in a CPU-internal register
load SS and ESP from TSS
push user SS
push user ESP
push user EFLAGS
push user CS
push user EIP
clear some EFLAGS bits (XXX what?)
set CS and EIP from IDT descriptor's segment selector and offset

Q: does INT really need all those steps?
e.g. why does it save SS and ESP?

what's the current CPL?
print $cs
why can't user code abuse INT to get privilege?

xv6 details:
vectors.S
tvinit() (trap.c) sets up IDT during boot
switchuvm() (vm.c) sets ss and esp in TSS

x/6x $esp
Q: which stack?
what's on it?
user registers: fake error, eip, cs, eflags, esp, ss

x/3i vector64
why push 0x40?
why not have IDT point directly to alltraps?

stepi to pushl %esp (trapasm.S, just before call trap)

x/19x $esp
these are all saved *user* registers
compare with struct trapframe, x86.h
ss // hw pushes these:
esp
eflags

```
  cs
  eip
  (error)
  trapno   // vector pushes this
  ds       // alltraps pushes these:
  es
  fs
  gs
  eax
  ecx
  edx
  ebx
  xxesp
  ebp
  esi
  edi
```

Q: where does trap(tf) argument come from?

Q: where did tf->trapno come from?

*** system call handling

if T_SYSCALL (0x40), trap() calls syscall()
  syscall() gets system call number from tf->eax
    Q: where was %eax set?
  Q: where are the system call arguments?
  sys_sbrk() fetches system call argument from user stack
    pushed by user-level C call to sbrk()
    via tf->esp
    argint()

syscall() puts sys_sbrk() return value in tf->eax

syscall() returns to trap()
  trap() returns to alltraps (trapasm.S)

*** kernel -> user

use "finish" to return to trapasm.S
si (until popal)
x/19x $esp

Q: what has changed in the trapframe?

si until iret
x/5x $esp
  eip cs eflags esp ss

si (into user space)

Q: where are we now?
Q: what are we returning from?

x/4x $esp
Q: what stack is that?
Q: what's on the stack?

*** other points about interrupts/exceptions

Faults, like page fault and divide by zero, work much the same, but to different vectors.

The kernel handles some exceptions internally, e.g. lazy-allocate page fault.

The program can arrange to get control after some exception -- via UNIX signal handlers.

Faults, interrupts, &c can occur while the kernel is running
  old CPL == new CPL
  h/w doesn't switch stacks
  h/w doesn't push old esp/ss
  so trapframe is a bit different
  you can tell: look at old CPL in low bits of tf->cs

device interrupts
  hardware generates them
  examples: timer, disk, console
  vector per device

example device: timer
  trap.c, look for IRQ_TIMER
  add a cprintf, you'll see it's called a lot

*** fork

a process's kernel stack is originally set up in fork()

we want to know setup of child
  user stack, registers, EIP
  kernel stack, registers, EIP

struct proc, in proc.h
  each has to be initialized in child

allocproc()
  kstack allocation
  space for trapframe
  *sp = trapret
  space for context
  context->eip = forkret

fork()
  copyuvm() -- copy user stack, instructions, heap
  *np->tf = *proc->tf -- copy user registers
  np->tf->eax = 0 -- child return value

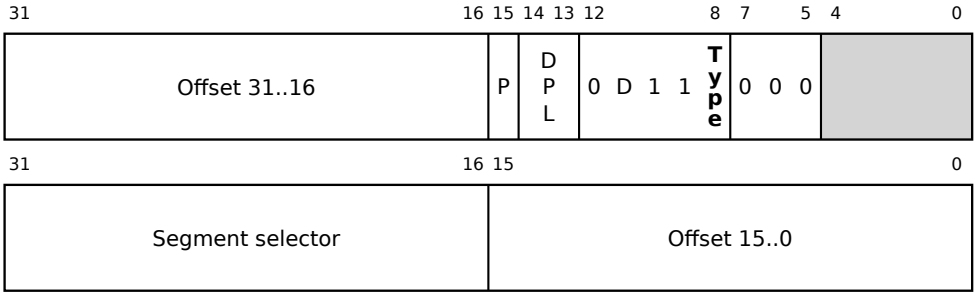
MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

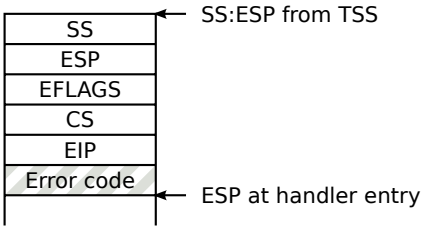
For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

Interrupt/trap gate

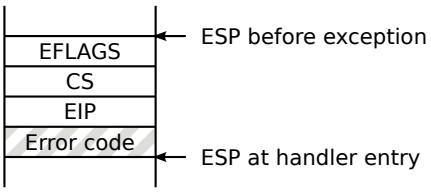


Type	0=Interrupt gate 1=Trap gate	P	Present
Selector	Destination CS	DPL	Descriptor privilege level (CPL required to invoke gate)
Offset	Destination IP or EIP	D	Size of gate (0=16-bits, 1=32-bits)

Exception stack (with privilege change)



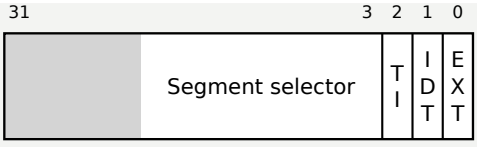
Exception stack (without privilege change)



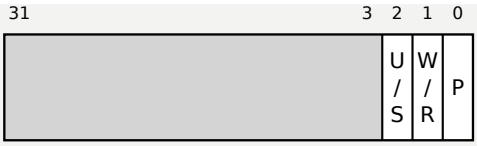
Vector	Description	Type	Error code
0	Divide error	Fault	No
1	Reserved		
2	Non-maskable interrupt	Interrupt	No
3	Breakpoint	Trap	No
4	Overflow	Trap	No
5	BOUND range exceeded	Fault	No
6	Invalid/undefined opcode	Fault	No
7	No math coprocessor	Fault	No
8	Double fault	Abort	Zero
9	Reserved		
10	Invalid TSS	Fault	Yes
11	Segment not present	Fault	Yes
12	Stack-segment fault	Fault	Yes
13	General protection	Fault	Yes
14	Page fault	Fault	Yes
15	Reserved		No
16	x87 FPU error	Fault	No
17	Alignment check	Fault	Zero
18	Machine check	Abort	No
19	SIMD FP exception	Fault	No
20-31	Reserved		
32-255	User defined interrupts	Interrupt	No

Exception types

Fault	Faulting instruction not executed CS:EIP is the faulting instruction
Trap	Trapping instruction executed CS:EIP is the next instruction
Abort	Location is imprecise; cannot safely resume execution



TI	0=GDT, 1=LDT
IDT	0=GDT/LDT, 1=IDT
EXT	External event



P	0=Non-present page 1=Protection-violation
W/R	Cause (0=Read, 1=Write)
U/S	Mode (0=Supervisor, 1=User)

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

-*- mode: org -*-
#+STARTUP: indent

* <2011-09-26 Mon>: Lec: Intro to concurrency
Students read text, and do homework assignment
Last lecture didn't cover device interrupts

* Plan: concurrency
** SMP
** Devices
** locks
** interrupts
** more sophisticated techniques (e.g., lock-free later in term)

* abstract SMP architecture
** processors with one shared memory
** devices
** interrupts processed in parallel

* Race conditions
** multiple processes adding to the disk queue
** simplified code:

```
struct List {
  int data;
  struct List *next;
};

List *list = 0;

insert(int data) {
  List *l = new List;
  l->data = data;
  l->next = list;  // A
  list = l;        // B
}
```

Whoever wrote this code probably believed it was correct, in the sense that if the list starts out correct, a call to insert() will yield a new list that has the old elements plus the new one. And if you call insert(), and then insert() again, the list should have two new elements. Most programmers write code that's only correct under serial execution -- in this case, insert() is only correct if there is only one insert() executing at a time.

What could go wrong with concurrent calls to insert(), as might happen on a multiprocessor? Suppose two different processors both call <code>insert()</code> at the same time. If the two processors execute A and B interleaved, then we end up with an incorrect list. To see that this is the case, draw out the list after the sequence A1 (statement A executed by processor 1), A2 (statement A executed by processor 2), B2, and B1.

* Goals
serialize inserts on same list

serialize inserts & deletes on same list
ops on different lists in parallel

* Popular tool: lock
** lock protects an invariant

** Avoiding the list race:
Lock list_lock;   // one per list

```
insert(int data){
  List *l = new List;
  l->data = data;

  acquire(&list_lock);

  l->next = list ; // A
  list = l;       // B

  release(&list_lock);
}
```

*** code between acquire/release: critical section, or atomic section

** Return to device driver
*** One lock for all disk devices
*** Invariants
The disk hardware can only execute one read or write at a time.
Only one process should be inserting or deleting from ide_queue at a time.
Only one process should be commanding the IDE hardware (via inb/outb
instructions) at a time.
*** iderw: why no locks around first instruction of the function?
*** Device driver has its own concurrency invariants:
processor shouldn't write/read buffer when disk is using it
how such invariants enforced? (answer: by careful programming...)

** How hard is it to produce the list race
*** let's try with stressfs
*** must read because logging system serializes writes

* Implementing lock and release
** use atomic instructions (e.g., xchg)
For example, the x86 <tt>xchg %eax, addr</tt> instruction
does the following:
freeze other CPUs' memory activity for address addr
temp := *addr
*addr := %eax
%eax = temp
un-freeze other memory activity
** xv6 lock implementation
instruction reordering
** lock is a *spin lock*

* Design issues
** Granularity (e.g., BKL)

Should we have one lock for kernel (this year your JOS will do this ...)
Should we have one lock per device? (ide)
** Spinning versus sleeping
Spin locks not good for waiting until device driver returns
xv6: separate primitive to go to sleep
some kernel automatically switch to sleeping
** locks and interrupts
ide disk generates interrupt when disk operation completes
interrupt causes ideintr to run
ideintr acquires lock?
  deadlock?
  give interrupt handler lock? (recursive locks)
xv6: critical sections have no interrupts turned on
** locks and modularity
*** locks are part of the spec of a module (e.g., idestart assumes caller got lock)
```
  move(l1, l2) {
    e = del(l1)
    insert(l2, e)
  }
```
various problems: there is a time when it is observable that e is neither list
```
  move(l1, l2) {
    acquire(l1.lock);
    acquire(l2.lock);
    e = del(l1)
    insert(l2, e)
    release(l1.lock)
    release(l2.lock)
  }
```
*** recursive locks are a bad idea
ideintr should certainly not use that instead of disabling interrupts!
** lock ordering
iderw: sleep acquires plock
--> never acquire plock and then ide_lock

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

-*- mode: org -*-
#+STARTUP: indent

how to run connect to qemu from gdb
how to pass a break point multiple times (c n)

* <2011-10-03 Mon>: processes, threads, and scheduling

* Plan:
 process
 threads
 scheduling

* Process:
** abstract virtual machine
provides the illusion to application of a dedicated computer, but an abstract one
convenient for application developer
one process cannot effect another accidentally
** API:
fork
exec
exit
wait
kill
sbrk
getpid

* Problem: more processes than processors
** xv6 picture:
1 user thread and 1 kernel thread per process
1 scheduler thread per processor
n processors
** terms
*** a process: address space plus one or more threads
*** a thread: thread of execution
kernel thread: thread running in kernel mode
user thread: thread running in user mode
*** thread of execution:
an abstraction that contains enough state of a running program that it can be
stopped and resumed
xv6 API: yield, swtch

* Goals for solution:
- Switching transparent to user threads
- User thread cannot hog a processor (kernel thread assumed to be correct, so
  not a goal)

* Overview of switch between two user threads
** user threads
- User -> kernel transition
- kernel -> kernel switch
- kernel -> User transition
** guaranteed U->K transitions

- timing interrupt every 100 ms
- switches to different kernel thread on yield
- the different kernel thread returns to a different user thread

* Challenges in implementing:
** Opaque code ("You are not supposed to understand this")
** Concurrency (several processors switching between threads)
** Terminating a thread, always need a valid stack

* Xv6 design
One scheduler thread per processor
Scheduling organized as co-routines
Scheduler thread performs cleanup

* Code
** Forced switching:
*** demo of two processes who don't invoke system calls
**** look at process states
*** clock interrupt
lapic.c for SMP
timer.c for uniprocessor
*** walk through what xv6 does to guarantee switching
breakpoint in trap
get hog running (c 100)
look at tf, in particular tf->eip
look at tf->trapno (timer interrupt), gets to yield
get to swtch, look at contexts (p /x *cpus[0]->scheduler)
look at eip before return from swtch (we switched to scheduler thread)
scheduler: switches to selected thread (set b proc.c:278)
will return user space
what is the scheduling policy?
will the thread that called yield run immediately again?
** Concurrency
 - plock held across swtch; why?
   yield: p is set runnable, p must complete switch before another scheduler choses p
 - hard to reason about; coroutine style helps
 - can two schedulers select the same runnable process?
 - why does scheduler release after loop, and re-acquire it immediately? (run with interrupts!)
** Thread clean up
  - let's look at kill: can we clean up killed process? (no: it might be running,
  holding locks etc.)
    before returning to user space: process kills itself by calling exit
  - let's look at exit; can thread delete its stack? (no: it has to switch off
    it!)
  - wait() does the cleanup

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

-*- mode: org -*-
#+STARTUP: indent

* plan
- context switching solution
- sequence coordination
- xv6's implementation
- in-class exercise

* context switching solution
show solution

* big picture:
- Multiple threads executing in the kernel
- sharing memory, devices, and various data structures.
- locks to protect invariants
  one outstanding disk request
  one scheduler selecting a thread to run
- show context switching pattern in kernel

* sequence coordination:
- how to arrange for threads to wait for each other to do
  wait for disk interrupt to complete
  wait for pipe readers to make space in pipe
  wait for child to exit
  wait for block to use

* straw man solution: spin
waste CPU cycles if need to spin for long time

* beter solution: sleep when waiting
tricky: lost wakeup
- case study: iderw()
- why does sleep take the ide lock as argument?
  comment acquire/relase in sleep/wakeup
  sleep misses wakeup; deadlock
  ideintr() runs before we to sleep

* sleep arranges making going to sleep atomic:
first hold ptable lock
set SLEEP
then release the lock argument
but requires API change: sleep takes a lock argument

* Another example: pipe
what is the race if sleep didn't take p->lock as argument?

* Real kernels also deal with receiving a ctrl-C, e.g., in sleep.
This is messy because a process could sleep somewhere deep in the kernel
A signal forces it out of sleep
But when it comes out of the sleep it is not because the condition it is waiting on is true.
A common approach is use longjmp (unwind the stack), and retry the system call.
xv6 doesn't do this; but handles kill signal when sleeping on pipe by checking

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2011 Lecture 9: File System

topic: file systems
  what they are
  how the xv6 file system works
  intro to larger topics

FS goals?
  durable storage
  multiplexing
  sharing
  organization

why are file systems interesting?
  they are a critical (and fruitful) piece of many stories
  performance
  security
  fault tolerance
  O/S API design
  user interfaces
  and you will implement one for JOS

high-level choices made by UNIX (and xv6)
  granularity: files (vs virtual disk, DB)
  content: byte array (vs 80-byte records, BTree)
  naming: human-readable (vs object IDs)
  organization: name hierarchy
  synchronization: none (vs locking, versions)

what's the API?
  fd = open("x/y", O_CREATE);
  write(fd, "abc", 3);
  link("x/y", "x/z");
  unlink("x/y");

FS abstraction turns out to be useful
  pipes
  devices /dev/console
  Linux /proc
  /afs
  Plan 9
  point: apps don't have to know about each of these objects separately
    write app to general FS/FD interface, works for many underlying things

a few implications of the API:
  fd refers to something
    that is preserved even if file name changes
    or if file is deleted while open!
  a file can have multiple links
    i.e. occur in multiple directories
    no one of those occurences is special
    so cannot store info about file in the directory
  thus:

a file exists independent of its names
called an "inode"
inode must have link count (tells us when to free)
inode must have count of open FDs
inode deallocation deferred until last link and FD are gone

let's talk about xv6

FS software layers
- system calls
- name ops | file descriptors
- inode ops
- inode cache (really just active inodes)
- transactions
- buffer cache
- disk driver

on-disk layout
- we will view disk as linear array of 512-byte sectors
  - though, when thinking about performance, remember concentric tracks
- 0: unused
- 1: super block (size, ninodes)
- 2: array of inodes, packed into blocks
- X: block in-used bitmap (0=free, 1=inuse)
- Y: file/dir content blocks
- Z: log for transactions
- end of disk

on-disk inode
- type (free, file, directory, device)
- nlink
- size
- addrs[12+1]

direct and indirect blocks

each i-node has an i-number
- easy to turn i-number into inode
- location on disk: sector 2 + 64*inum

directory contents
- directory much like a file
  - but user can't directly write
- content is array of dirents
- dirent:
  - inum
  - 14-byte file name
- dirent is free if inum is zero

you should view FS as an on-disk data structure
- [tree: dirs, inodes, blocks]
- two allocation pools: inodes and blocks

let's look at xv6 in action

focus on disk writes, as in homework
illustrate on-disk data structures via how updated

[hand out the following]

Q: how does xv6 create a file?

```
$ echo > a
log_write 4 ialloc (44, from create 54)
log_write 4 iupdate (44, from create 54)
log_write 29 writei (47, from dirlink 48, from create 54)
log_write 2 iupdate
```

Q: what is writei writing?

Q: what is the 2nd iupdate about?

Q: what if there are concurrent calls to ialloc?
will they get the same inode?
note bread / bwrite / brelse in ialloc
bread sheet 39
diagram of block cache
focus on B_BUSY and sleep()
Q: why goto loop?

Q: how does xv6 write data to a file?

```
$ echo x > a
log_write 28 balloc (43, from bmap 46, from writei 47)
log_write 417 bzero
log_write 417 writei
log_write 4 iupdate
log_write 417 writei
log_write 4 iupdate
```

Q: why the iupdate?

Q: why *two* iupdates?

Q: how does xv6 delete a file?

```
$ rm a
log_write 29 writei
log_write 4 iupdate
log_write 28 bfree
log_write 4 iupdate
log_write 4 iupdate
```

Q: what's in block 29?

Q: what are the iupdates doing?

Q: what is the block cache replacement policy?
bget and brelse, sheet 39

Q: is that the best replacement policy?

Q: when does xv6 write user data to disk?
  writei 47, bwrite 39, iderw 38 (which sleeps)

Q: is that a good write policy?
  performance?
  correctness? what's the danger?

Q: when does xv6 write meta-data to disk?
  same policy as user data
  is that a good meta-data write policy?
    performance
    correctness (order)
  tune in next lecture...

Q: what if lots of processes need to read the disk? who goes first?
  iderw 38 appends to idequeue
  idestart looks at head of list
  ideintr pops head, starts next
  so FIFO

Q: is FIFO a good disk scheduling policy?
  priority to interactive programs?
  elevator sort?

Q: how fast can an xv6 application read big files?
  contiguous blocks?
  blow a rotation -- no prefetch?

Q: why does it make sense to have a double copy of I/O?
  disk to buffer cache
  buffer cache to user space
  can we fix it to get better performance?

Q: how much RAM should we dedicate to disk buffers?

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2011 Lecture 10: Crash Recovery, Logging

what is crash recovery?
  you're writing the file system
  then the power fails
  you reboot
  is your file system still useable?

the main problem:
  crash during multi-step operation
  leaves FS invariants violated
  can lead to ugly FS corruption

examples:
  create:
    new dirent
    allocate file inode
    crash: dirent points to free inode -- disaster!
    crash: inode not free but not used -- not so bad
  write:
    block content
    inode addrs[] and len
    indirect block
    block free bitmap
    crash: inode refers to free block -- disaster!
    crash: block not free but not used -- not so bad
  unlink:
    block free bitmaps
    free inode
    erase dirent

what can we hope for?
  after rebooting and running recovery code
  1. FS internal invariants maintained
     e.g., no block is both in free list and in a file
  2. all but last few operations preserved on disk
     e.g., data I wrote yesterday are preserved
     user might have to check last few operations
  3. no order anomalies
     echo 99 > result ; echo done > status

simplifying assumption: disk is fail-stop
  disk executes the writes FS sends it, and does nothing else
    perhaps doesn't perform the very last write
  thus:
    no wild writes
    no decay of sectors

correctness and performance often conflict
  safety => write to disk ASAP
  speed => don't write the disk (batch, write-back cache, sort by track, &c)

we'll discuss two approaches:

synchronous meta-data update + fsck
logging (xv6 and linux ext3)

synchronous meta-data update
an old approach to crash recovery
simple, slow, incomplete

most problem cases look like dangling references
inode -> free block
dirent -> free inode

idea: always initialize *on disk* before creating reference
implement by doing the initialization write,
waiting for it to complete,
and only then doing the referencing write
"synchronous writes"

example: file creation
what's the right order of synchronous writes?
1. mark inode as allocated
2. create directory entry

example: file deletion
1. erase directory entry
2. erase inode addrs[], mark as free
3. mark blocks free

example: rename() (not in xv6)
between directories, i.e. mv d1/x d2/y
1. create new dirent
2. erase old dirent
or the other way around?
probably safest to create then erase!

what will be true after crash+reboot?
all completed sys calls guaranteed visible on disk
reachable part of FS will be mostly correct
except interrupted rename leaves file in both directories!
blocks and inodes may be unreferenced but not marked free

so: sync meta-data update system needs to check at reboot
to free unreferenced inodes and blocks
descend dir tree from root, remembering all i-numbers and block #s seen
mark everthing else free
probably have to punt on interrupted rename()

many kinds of UNIX used sync writes until 10 years ago

problems with synchronous meta-data update
very slow during normal operation
very slow during recovery

how long would fsck take?
a read from a random place on disk takes about 10 milliseconds

descending the directory hierarchy might involve a random read per inode
so maybe (n-inodes / 100) seconds?
faster if you read all inodes (and dir blocks) sequentially,
then descend hierarchy in memory
my server: fsck takes 10 minutes per 70GB disk w/ 2 million inodes
clearly reading many inodes sequentially, not seeking
still a long time, probably linear in disk size

ordinary performance of sync meta-data update?
creating a file and writing a few bytes takes 8 writes, probably 80 ms
(ialloc, init inode, write dirent, alloc data block, add to inode,
write data, set length in inode, one other mystery write to data)
so can create only about a dozen small files per second!
think about un-tar or rm *

how to get better performance?
RAM is cheap
disk sequential throughput is high, 50 MB/sec
(maybe someday solid state disks will change the landscape)
we'll talk about big memory, then sequential disk throughput

why not use a big write-back disk cache?
no sync meta-data update
operations *only* modify in-memory disk cache (no disk write)
so creat(), unlink(), write() &c return almost immediately
bufs written to disk later
if cache is full, write LRU dirty block
write all dirty blocks every 30 seconds, to limit loss if crash
this is how old Linux EXT2 file system worked

would write-back cache improve performance? why, exactly?
after all, you have to write the disk in the end anyway

what can go wrong w/ write-back cache?
example: unlink() followed by create()
an existing file x with some content, all safely on disk
one user runs unlink(x)
1. delete x's dir entry **
2. put blocks in free bitmap
3. mark x's inode free
another user then runs create(y)
4. allocate a free inode
5. initialize the inode to be in-use and zero-length
6. create y's directory entry **
again, all writes initially just to disk buffer cache
suppose only ** writes forced to disk, then crash
what is the problem?
can fsck detect and fix this?

how can we get both speed and safety?
write only to cache
somehow remember relationships among writes
e.g. don't send #1 to disk w/o #2 and #3

most popular solution: logging (== journaling)
  goal: atomic system calls w.r.t. crashes
  goal: fast recovery (no hour-long fsck)
  goal: speed of write-back cache for normal operations

will introduce logging in two steps
  first xv6's log, which only provides safety
  then Linux EXT3, which is also fast

the basic idea behind logging
  you want atomicity: all of a system call's writes, or none
    let's call an atomic operation a "transaction"
  record all writes the sys call *will* do in the log
  then record "done"
  then do the writes
  on crash+recovery:
    if "done" in log, replay all writes in log
    if no "done", ignore log
  this is a WRITE-AHEAD LOG

xv6's simple logging
  [diagram: buffer cache, FS tree on disk, log on disk]
  FS has a log on disk
  syscall:
    begin_trans()
      bp = bread()
      bp->data[] = ...
      log_write(bp)
      more writes ...
    commit_trans()
  begin_trans:
    need to indicate which group of writes must be atomic!
    lock -- xv6 allows only one transaction at a time
  log_write:
    record sector #
    append buffer content to log
    leave modified block in buffer cache (but do not write)
  commit_trans():
    record "done" and sector #s in log
    do the writes
    erase "done" from log
  recovery:
    if log says "done":
      copy blocks from log to real locations on disk

let's look at the code:
  sys_unlink, sheet 54
    begin_trans before ilock to avoid deadlock
      then error checks, which need the inode lock
      on err, commit empty transaction
    writei of dirent
    iupdate and iunlockput of file
      thus freeing of blocks, erasing of addrs[], freeing inode
    commit_trans

  begin_trans, sheet 41
    why only one transaction at a time?
  log_write
  commit_trans
    write_head
    install_trans
  recover_from_log

let's look at today's homework
the log header is at 1014
$ rm README
bwrite sector 1015 -- 29, writei
bwrite sector 1016 -- 2, iupdate
bwrite sector 1017 -- 28, bfree
bwrite sector 1017 -- 28, bfree
bwrite sector 1017 -- 28, bfree
bwrite sector 1017 -- 28, bfree
bwrite sector 1016 -- 2, iupdate
bwrite sector 1016 -- 2, iupdate
bwrite sector 1014 -- log header <-- commit point
bwrite sector 29   -- dir content
bwrite sector 2   -- root and file inodes
bwrite sector 28   -- free bitmap
bwrite sector 1014 -- erase transaction

what's wrong with xv6's logging?
  only one transaction at a time
    two system calls might be modifying different parts of the FS
  log traffic will be huge: every operation is many records
  logs whole blocks even if only a few bytes written
  eager write to log -- slow
  eager write to real location -- slow
  every block written twice
  trouble with operations that don't fit in the log
    unlink might dirty many blocks while truncating file

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2011 Lecture 11: Linux ext3 crash recovery

topic
  crash recovery
    crash may interrupt a multi-disk-write operation
    leave file system in an unuseable state
  most common solution: logging
  last lecture: xv6 log -- simple but slow
  today: Linux ext3 log -- fast
  theme: speed vs safety
    speed: don't write the disk
    safety: write the disk ASAP

example problem:
  appending to a file
  two writes:
    mark block non-free in bitmap
    add block # to inode addrs[] array
  we want atomicity: both or neither
  so we cannot do them one at a time

why logging?
 goal: atomic system calls w.r.t. crashes
 goal: fast recovery (no hour-long fsck)
 goal: speed of write-back cache for normal operations

review of xv6 logging
  [diagram: buffer cache, FS tree on disk, log on disk]
  log "header" block and data blocks
  each system call is a transaction
    begin_trans, commit_trans
  only one transaction at a time
  syscall writes in buffer cache
  each written block appended to log
    but NOT yet written to "home" location
    "write-ahead log"
    preserve old copy until sure we can commit
  on commit:
    write "done" and block #s to header block
  then write modified blocks to home locations
  then erase "done" from header blocks
  recovery:
    if log says "done":
      copy blocks from log to real locations on disk

what's wrong with xv6's logging? it is slow!
  only one transaction at a time
    two system calls might be modifying different parts of the FS
  synchronous write to on-disk log
    each write takes one disk rotation time
    commit takes a nother
    a file create/delete involves around 10 writes
    thus 100 ms per create/delete -- very slow!

tiny update -> whole block write
creating a file only dirties a few dozen bytes
but produces many kilobytes of log writes
synchronous writes to home locations after commit
i.e. write-through, not write-back
makes poor use of in-memory disk cache

how can we get both performance and safety?
we'd like system calls to proceed at in-memory speeds
using write-back disk cache
i.e. have typical system call complete w/o actual disk writes

Linux's ext3 design
case study of the details required to add logging to a file system
Stephen Tweedie 2000 talk transcript "EXT3, Journaling Filesystem"
ext3 adds a log to ext2, a previous xv6-like log-less file system
has many modes, I'll start with "journaled data"
log contains both metadata and file content blocks

ext3 structures:
in-memory write-back block cache
in-memory list of blocks to be logged, per-transaction
on-disk FS
on-disk circular log file

what's in the ext3 log?
superblock: starting offset and starting seq #
descriptor blocks: magic, seq, block #s
data blocks (as described by descriptor)
commit blocks: magic, seq

how does ext3 get good performance despite logging entire blocks?
batches many syscalls per commit
defers copying cache block to log until it commits log to disk
hopes multiple sycalls modified same block
thus many syscalls, but only one copy of block in log
"write absorbtion"

sys call:
h = start()
get(h, block #)
warn logging system we'll modify cached block
added to list of blocks to be logged
prevent writing block to disk until after xaction commits
modify the blocks in the cache
stop(h)
guarantee: all or none
stop() does *not* cause a commit
notice that it's pretty easy to add log calls to existing code

ext3 transaction
[circle set of cache blocks in this xaction]
while "open", adds new syscall handles, and remembers their block #s
only one open transaction at a time

ext3 commits current transaction every few seconds (or fsync())

committing a transaction to disk
open a new transaction, for subsequent syscalls
mark transaction as done
wait for in-progress syscalls to stop()
(maybe it starts writing blocks, then waits, then writes again if needed)
write descriptor to log on disk w/ list of block #s
write each block from cache to log on disk
wait for all log writes to finish
append the commit record
now cached blocks allowed to go to homes on disk (but not forced)

is log correct if concurrent syscalls?
e.g. create of "a" and "b" in same directory
inode lock prevents race when updating directory
other stuff can be truly concurrent (touches different blocks in cache)
transaction combines updates of both system calls

what if syscall B reads uncommited result of syscall A?
A: echo hi > x
B: ls > y
could B commit before A, so that crash would reveal anomaly?
case 1: both in same xaction -- ok, both or neither
case 2: A in T1, B in T2 -- ok, A must commit first
case 3: B in T1, A in T2
could B see A's modification?
ext3 must wait for all ops in prev xaction to finish
before letting any in next start
so that ops in old xaction don't read modifications of next xaction

T2 starts while T1 is committing to log on disk
what if syscall in T2 wants to write block in prev xaction?
can't be allowed to write buffer that T1 is writing to disk
then new syscall's write would be part of T1
crash after T1 commit, before T2, would expose update
T2 gets a separate copy of the block to modify
T1 holds onto old copy to write to log
are there now *two* versions of the block in the buffer cache?
no, only the new one is in the buffer cache, the old one isn't
does old copy need to be written to FS on disk?
no: T2 will write it

performance?
create 100 small files in a directory
would take xv6 over 10 seconds (many disk writes per syscall)
repeated mods to same direntry, inode, bitmap blocks in cache
write absorbtion...
then one commit of a few metadata blocks plus 100 file blocks
how long to do a commit?
seq write of 100*4096 at 50 MB/sec: 10 ms
wait for disk to say writes are on disk
then write the commit record
that wastes one revolution, another 10 ms

modern disk interfaces can avoid wasted revolution

what if a crash?
crash may interrupt writing last xaction to log on disk
so disk may have a bunch of full xactions, then maybe one partial
may also have written some of block cache to disk
but only for fully committed xactions, not partial last one

how does recovery work
1. find the start and end of the log
log "superblock" at start of log file
log superblock has start offset and seq# of first transaction
scan until bad record or not the expected seq #
go back to last commit record
crash during commit -> last transaction ignored during recovery
2. replay all blocks through last complete xaction, in log order

what if block after last valid log block looks like a log descriptor?
perhaps left over from previous use of log? (seq...)
perhaps some file data happens to look like a descriptor? (magic #...)

when can ext3 free a transaction's log space?
after cached blocks have been written to FS on disk
free == advance log superblock's start pointer/seq

what if block in T1 has been dirtied in cache by T2?
can't write that block to FS on disk
note ext3 only does copy-on-write while T1 is commiting
after T1 commit, T2 dirties only block copy in cache
so can't free T1 until T2 commits, so block is in log
T2's logged block contains T1's changes

what if not enough free space in log for a syscall?
suppose we start adding syscall's blocks to T2
half way through, realize T2 won't fit on disk
we cannot commit T2, since syscall not done
can we free T1 to free up log space?
maybe not, due to previous issue, T2 maybe dirtied a block in T1
deadlock!

solution: reservations
syscall pre-declares how many block of log space it might need
block the sycall from starting until enough free space
may need to commit open transaction, then free older transaction
OK since reservations mean all started sys calls can complete + commit

ext3 not as immediately durable as xv6
creat() returns -> maybe data is not on disk! crash will undo it.
need fsync(fd) to force commit of current transaction, and wait
would ext3 have good performance if commit after every sys call?
would log many more blocks, no absorption
10 ms per syscall, rather than 0 ms
(Rethink the Sync addresses this problem)

no checksum in ext3 commit record
  disks usually have write caches and re-order writes, for performance
    sometimes hard to turn off (the disk lies)
    people often leave re-ordering enabled for speed, out of ignorance
  bad news if disk writes commit block before preceding stuff
  then recovery replays "descriptors" with random block #s!
  and writes them with random content!

ordered vs journaled
  journaling file content is slow, every data block written twice
  perhaps not needed to keep FS internally consistent
  can we just lazily write file content blocks?
  no:
    if metadata updated first, crash may leave file pointing
    to blocks with someone else's data
  ext3 ordered mode:
    write content block to disk before commiting inode w/ new block #
    thus won't see stale data if there's a crash
  most people use ext3 ordered mode

correctness challenges w/ ordered mode:
  A. rmdir, re-use block for file, ordered write of file,
       crash before rmdir or write committed
     now scribbled over the directory block
     fix: defer free of block until freeing operation forced to log on disk
  B. rmdir, commit, re-use block in file, ordered file write, commit,
       crash, replay rmdir
     file is left w/ directory content e.g. . and ..
     fix: revoke records, prevent log replay of a given block

final tidbit
  open a file, then unlink it
  unlink commits
  file is open, so unlink removes dir ent but doesn't free blocks
  crash
  nothing interesting in log to replay
  inode and blocks not on free list, also not reachably by any name
    will never be freed! oops
  solution: add inode to linked list starting from FS superblock
    commit that along with remove of dir ent
  recovery looks at that list, completes deletions

does ext3 fix the xv6 log performance problems?
  only one transaction at a time -- yes
  synchronous write to on-disk log -- yes, but 5-second window
  tiny update -> whole block write -- yes (indirectly)
  synchronous writes to home locations after commit -- yes

ext3 very successful
  but: no checksum -- ext4
  but: not efficient for applications that use fsync() -- next lecture

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

-*- mode: org -*-
#+STARTUP: indent

* <2011-11-05 Sat>, <2012-10-21 Sun>: L14: Operating System Organization

* Operating System Organization
students have completed building an exokernel in lab3 and lab4
user-space fork (copy-on-write)
sophisticated VM handling in libos
assumption in this lecture: one cannot change the kernel

* Plan: OS organization
** goals for a kernel interface
** monolithic
** microkernel
** exokernel
** little data, much opinion

* Goals for kernel
** Apps can use hardware resources
** Apps are easy convenient to write
** Apps are multiplexed (isolation and sharing)
** Few kernel crashes
** Kernel can evolve

* the topic is overall o/s design
lots of ways to structure an OS -- how to decide?
  what is the right kernel API?
looking for principles and approaches

* what's the traditional approach? (Linux, xv6)
kernel API ~ POSIX
virtualize some resources: cpu and memory
    simulate a dedicated cpu and memory system for each app
    why? it's a simple model for app programmers
abstract others: storage, network, IPC
    layer a sharable abstraction over h/w (file system, IP/TCP)

* example: virtualize the cpu
  goal: simulate a dedicated cpu for each process
    we want transparent CPU multiplexing
<   process need not think about how it interacts w/ other processes
  o/s runs different processes in turn, via clock interrupt
    clock means process doesn't need to do anything special to switch
    also prevents hogging
  how to achieve transparency?
    o/s saves state, then restores
  what does o/s save?
    eight regs, EIP, seg regs, eflags, page table base ptr
  where does o/s save it?
    o/s keeps per-process table of saved states
  the return from clock interrupt restores a *different* process's state
  the point: process doesn't have to worry about multiplexing!

this is the traditioal approach to virtualizing the CPU
what does the exokernel/JOS do?

* example: virtualize memory
idea: simulate a complete memory system for each process
so process has complete freedom how it uses that memory
doesn't have to worry about other processes
so addresses 0..2^32 all work, but refer to private memory
convenient: all programs can start at zero
and memory looks contiguous, good for large arrays &c
safe: can't even *name* another process's memory
again: traditional but we'll soon see it's a very limiting approach
really want apps to have more control than this style of VM implies

* level of indirection allows o/s to play other tricks
** demand paging:
process bigger than available physical memory?
"page-out" (write) pages to disk, mark PTEs invalid
if process tries to use one of those pages, MMU causes page fault
kern finds phys mem, page-in from disk, mark PTE valid
this works because apps use only a fraction of mem at a given time
need h/w valid flag, page faults, and re-startable instructions
** copy-on-write:
*** avoid copy implied by fork() -- won't be needed if exec()
make parent and child share the physical memory pages
*** if either writes, do the copy then
so need per-page write-protect flag
*** both of above are transparent to application
still thinks it has simple dedicated memory from 0..2^32
*** paging h/w has turned out to be one of the most fruitful ideas in o/s
you have been using it a lot in labs
** can we make it safe for apps to play these tricks?

* traditional organization: monolithic o/s
h/w, kernel, user
kernel is a big program: process ctl, vm, fs, network
all of kernel runs w/ full hardware privilege (very convenient)
good: easy for sub-systems to cooperate (e.g. paging and file system)
bad: interactions => complex, bugs are easy, no isolation within o/s
extensibility: dynamically-loadable modules, wait for next kernel release
** philosophy: convenience (for app or o/s programmer)
*** for any problem, either hide it from app, or add a new system call
(we need philosophy because there is not much science here)
may take a while for the new system call is added
*** very successful approach

* alterate organization: microkernel
philosophy: IPC and user-space servers
for any problem, make a new server, talk to it w/ RPC
h/w, kernel, server processes, apps
servers: VM, FS, TCP/IP, Print, Display
split up kernel sub-systems into server processes
some servers have privileged access to some h/w (e.g. FS and disks)
apps talk to them via IPC / RPC

kernel's main job: fast IPC
good: simple/efficient kernel, sub-systems isolated, enforced better modularity
bad: cross-sub-system optimization harder, lots of IPCs may be slow
extensibility: can replace servers
in the end, lots of good individual ideas, but overall plan didn't catch on for desktops/servers

* alternate organization: exokernel
philosophy: eliminate all abstractions
for any problem, expose h/w or info to app, let app do what it wants
h/w, kernel, environments, libOS, app
an exokernel would not provide address space, virtual cpu, file system, TCP
instead, give control to app:
phys pages, addr mappings, clock interrupts, disk i/o, net i/o
let app build nice address space if it wants, or not
should give aggressive apps much more flexibility
challenges:
how to multiplex cpu/mem/&c if you expose directly to apps?
how to get security/isolation despite apps having low-level control?
how to multiplex w/o understanding: disk (file system), incoming tcp pkts

* exokernel example: memory
what are the resources? (phys pages, mappings)
what does an app need to ask the kernel to do?
pa = AllocPage()
DeallocPage(pa)
TLBwr(va, pa)
and these kernel->app upcalls:
PageFault(va)
PleaseReleaseAPage()
what does o/s need to do to make multiplexing work?
ensure app only creates mappings to phys pages it owns
track what env owns what phys pages
decide which app to ask to give up a phys page when system runs out
that app gets to decide which of its pages

* simple example: shared memory
two processes want to share memory, for fast interaction
note traditional "virtual address space" doesn't allow for this
process a: pa = AllocPage()
put 0x5000 -> pa in private table
PageFault(0x5000) upcall -> TLBwr(0x5000, pa)
give pa to process b (need to tell exokernel...)
process b:
put 0x6000 -> pa in private table
...

* example cool thing you could do w/ exokernel-style memory
databases like to keep a cache of disk pages in memory
problem on traditional o/s:
assume an o/s with demand-paging to/from disk
if DB caches some disk data, and o/s needs a phys page,
o/s may page-out a DB page holding a cached disk block
but that's a waste of time: if DB knew, it could release phys
page w/o writing, and later read it back from DB file (not paging area)

1. exokernel needs phys mem for some other app
2. exokernel sends DB a PleaseReleaseAPage() upcall
3. DB picks a clean page, calls DeallocPage(pa)
4. OR DB picks dirty page, writes to disk, then DeallocPage(pa)

* exokernel example: cpu
  what does it mean to expose cpu to app?
    kernel tells app when it is taking away cpu
    kernel tells app when it gives cpu to app
  so if app is running and timer interrupt causes end of slice
    cpu jumps from app into kernel
    kernel jumps back into app at "please yield" upcall
    app saves state (registers, EIP, &c)
    app calls Yield()
  when kernel decides to resume app
    kernel jumps into app at "resume" upcall
    app restores those saved registers and EIP

* what cool things could an app do w/ exokernel-style cpu management?
  suppose time slice ends in the middle of
    acquire(lock);
    ...
    release(lock);
  you don't want the app to be holding the lock the whole time!
    then maybe other apps can't make forward progress
  so the "please yield" upcall can first complete the critical section

* fast RPC with direct cpu management
  how does traditional o/s let apps communicate?
    pipes (or sockets)
    picture: buffer in kernel, lots of copying and system calls
    RPC probably takes 8 kernel/user crossings (read()s and write()s)
  how does exokernel help?
    Yield() can take a target process argument
      almost a direct jump to an instruction in target process
      kernel allows only entries at approved locations in target
    kernel leaves regs alone, so can contain arguments
      (in constrast to traditional restore of target's registers)
    target app uses Yield() to return
    so only 4 crossings

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2012 Lecture 17: Language/OS Co-Design / Singularity

Why are we looking at this paper?
  completely different approach to isolation, protection
  language type-checking rather than hardware page protection

Singularity is a Microsoft Research experimental O/S
  many people, many papers, reasonably high profile
  stated goals:
    better robustness, security
    ground-up design w/ modern techniques

High level structure
  microkernel: SIP/thread mgmt, memory, IPC setup
    not so micro: 192 system calls (page 5)
  user-level service processes
    NIC, TCP/IP, FS, disk driver (sealing paper)
  not UNIX compatible

Most radical part of design:
  No hardware protection!
    Paging is turned off, so all memory visible to all instructions
    CPL=0, so can always run privileged instructions
  Instead: programming language protections

Why is that useful?
  Performance
  Fast process switching: no page table switch
  Fast system calls: CALL not INT
  Fast IPC: no copying
  Direct user program access to h/w, for e.g. device drivers
  Table 1 shows they are a lot faster at microbenchmarks

Q: why does no paging contribute to their main goal, robustness?

Remember what paging buys us:
  Protection
  Contiguous address space (starts at zero &c)
    Big arrays
    Contiguous stack
  Flexible address layout via mapping
  No fragmentation of physical memory
  Sharing/IPC via multiple mappings
  Tricks like copy-on-write fork, paging to disk

Challenges for no paging / CPL=0 design?
  Read/write only to appropriate memory
    And define what inappropriate means!
  Allow allocation and freeing of memory
  Allow interaction (IPC)
  But not too much entangling, so kill/exit can work

How to ensure SIP reads/writes only its own memory?

Q: why not have compiler generate check code before each load/store?
  speed, trust

The paper's approach:
 source
  compiler
 bytecodes
  install: verify and compile
 machine code
  trusted run-time
 running sip

Q: why not compile source to machine code?

Q: why verify/compile at install time? why not at run time -- JIT?

What properties does verification establish?
 Only use reachable pointers [draw diagram]
  only trusted runtime can create pointers
 So if kernel/runtime never supply out-of-SIP pointers
  verified SIP can only use its own memory

How does verification work?
 What does the verifier have to check?
  A. Don't invent or modify pointers
  B. Don't change mind about type
    Would allow violation of A, e.g. interpret int as pointer
  C. Don't use after free
    Re-use might change type, violate B
    Enforced with GC (and exchange heap linearity)
  D. Don't use uninitialized variables
  E. In general, don't trick the verifier
 Example bytecodes:
    R0 <- new SomeClass;
    jmp L1
    ...
    R0 <- 1000
    jmp L1
    ...
  L1:
    mov (R0) -> R1
  Q: is this code OK?
  verifier tries to deduce type for every register
    by pretending to execute along each code path
    requires that all paths to a reg use result in same type
    check that all reg uses OK for type
  verifying the example:
    R0 has type SomeClass at first jmp to L1
    R0 has type integer at second jmp to L1
    so verifier would reject this code

Bytecode verification seems to do *more* than Singularity needs
 e.g. cooking up pointers might be OK, as long as within SIP's memory

so verifier may forbid some OK programs
this style of verification is off-the-shelf
enforcing exactly what Singularity needs is not
Singularity may actually need full verification
can't allow jump to data, even if data is in process's memory
since then executing unchecked code

What parts of verification scheme are trusted vs untrusted?
That is:
All s/w has bugs
Trusted s/w: if it has bugs, it can crash Singularity or wreck other SIPs
Untrusted s/w: if it has bugs, can only wreck itself
Source?
Compiler?
Compiler output?
Verifier?
Machine code output of bytecode compiler?
Runtime / GC?

IPC: what would we want?
shared memory for efficiency
send complex data structures
but still have isolation, type checking

How do SIPs communicate?
IPC messages
"exchange heap" -- memory shared among all SIPs
thus zero-copy -- efficient
msgs can have pointers &c
thus can send complex data structures
each receiver has a queue in the exchange heap
send() system call to wake up receiving SIP
receiver blocks in recv() sys call, then checks queue

Q: dangers of shared-memory exchange heap?
write someone else's message
send the wrong type of data
modify my msg while you are reading it
use up all exchange heap memory and don't free

How do they prevent abuse via exchange heap?
verifier ensures SIP can only use ptrs someone gives it
i.e. only if you allocated mem, or found it in your recv queue
verifier ensures SIP bytecodes keep only one ptr to anything in exchange heap
never e.g. two
and that SIP doesn't keep ptr after send()
single-ptr rule helps here
verifier knows when last ptr goes away
via send
via making another exchange heap obj point to it
via delete
single ptr rule prevents change-after-send
and also ensures delete when done
delete is explicit, no GC, but it's OK

since verifier guarantees only a single ptr to each block
runtime maintains owning-SIP entry in each exchg heap block
updates on send() &c
used to clean up in exit()

Limitations of exchange heap idea
IPC can't carry existing language object -- not in exchange heap
Single-pointer rule limits the code you write
Need to use different types/functions for exchange heap data

What are channel contracts for? Section 2.2
Are they just nice to have, or do other parts of Singularity rely on them?
The type signatures clearly are important
they probably mesh with verified language types
perhaps you can't talk to a SIP that isn't verified to follow contract
The state machine part guarantees finite queues, no blocking send().
and also catches protocol implementation errors
e.g. sending msg when not expected

How do system calls into the kernel work?
INT? CALL?
what stack?
can a SIP pass pointers to kernel?
how does SIP GC know to not examine kernel part of stack?

Q: SIP allocates single pages -- how to have stack > 4096 bytes?

Q: How to have array > 4096 bytes?

2.1 says SIPs are "sealed"
Outlawed: JIT, dynamic library loader, self-modifying, debugger (?)
Q: Why is this important?
no code insertion attacks
maybe easier to reason about correctness
maybe easier to optimize, inline
e.g. delete unused functions
SIP can be a security principle, own files

You could put an interpreter in a SIP to evade ban on self-modifying code
Would that cause trouble?

Why not use a Java VM as your operating system?
Java has verification -- you can't make up pointers &c
Could have a Java thread for each running application

Singularity vs Java VM:
One SIP can *never* affect another SIP's memory
Not even with IPC
Would be easy to have such bugs w/ interacting Java threads
Exiting/killing a SIP releases all resources
Java must at least wait for a GC
SIPs let every process have its own language run-time, GC scheme, &c

What should the evaluation show?

What were their goals?
To gain robustness -- perhaps better than paging / CPL=3
To re-examine traditional design choices

What does the evaluation show?
Mostly about performance

Table 1 shows microbenchmark performance
10x reduction in sys calls -- why?
2x faster thread switch -- why?
5x faster IPC -- why?
2x-10x faster process creation -- why?

Figure 5: unsafe code tax
How much do they gain by static verification rather than run-time (or h/w) checks?
Simple file reading benchmark -- client SIP, file server SIP, device driver SIP
Figure 5 compare run-times; lower is better
physical memory -- Singularity (no paging, CPL=0, static verification, &c)
No runtime checks -- Singularity but no array bounds checking
Add 4KB pages -- paging enabled, but single page table, all CPL=0
separate domain -- separate page table for one of the SIPs, so switching costs
ring 3 -- CPL=3 thus INT costs (for just one of the SIPs)
full microkernel -- pgtable+INT for each of three SIPs
Figure 5 is useful: shows costs of various x86 features

What did we learn?
Are 1960s and 70s techniques now inadequate?
Should we use verification &c instead of paging hardware?
We *did* learn how to build O/S w/o paging -- very interesting!

A few open questions:
What are manifests for?
Can IPC carry a capability? How does kernel learn?
Does IPC receiver have to check msg format at run-time?
Why is the exchange heap data reference counted? When can the count be > 1?
When is it OK for SIP user code to execute a CPL=0 privileged instruction?

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2012 Lecture 18: Scalable Locks

Plan:
  cost of spinlocks -- non-scalable
  effect on real systems
  scalable locks

Why this topic?
  Figure 2 in the paper -- disaster! (details later)
  the locks themselves are ruining performance
    rather than letting us harness multi-core to improve performance
  this "non-scalable lock" phenomenon is important
  why it happens is interesting and worth understanding
  the solutions are clever exercises in parallel programming

the problem is interaction of locks w/ multi-core caching
  so let's look at the details

back in the locking lecture, we had a fairly simple model of multiple cores
  cores, shared bus, RAM
  to implement acquire, x86's xchg instruction locked the bus
    provided atomicity for xchg

real computers are much more complex
  bus, RAM quite slow compared to core speed
  per-core cache to compensate
  hit: a few cycles
  RAM: 100s of cycles

how to ensure caches aren't stale?
  core 1 reads+caches x=10, core 2 writes x=11, core 1 reads x=?

answer:
  "cache coherence protocol"
  ensures that each read sees the latest write
    actually more subtle; look up "sequential consistency"

how does cache coherence work?
  many schemes, here's a simple one
  each cache line: state, address, 64 bytes of data
  states: Modified, Shared, Invalid [MSI]
  cores exchange messages as they read and write

messages (much simplified)
  invalidate(addr): delete from your cache
  find(addr): does any core have a copy?
  all msgs are broadcast to all cores

how do the cores coordinate with each other?
  I + local read -> find, S
  I + local write -> find, inval, M
  S + local read -> S
  S + local write -> inval, M

S + recv inval -> I
S + recv find -> nothing, S
M + recv inval -> I
M + recv find -> reply, S

can read w/o bus traffic if already S
can write w/o bus traffic if already M
"write-back"

compatibility of states between 2 cores:

		core1		
		M	S	I
core2	M	-	-	+
	S	-	+	+
	I	+	+	+

invariant: for each line, at most one core in M
invariant: for each line, either one M or many S, never both

Q: what patterns of use benefit from this coherence scheme?
read-only data (every cache can have a copy)
data written multiple times by one core (M gives exclusive use, cheap writes)

other plans are possible
e.g. writes update copies rather than invalidating
but "write-invalidate" seems generally the best

Real hardware uses much more clever schemes
mesh of links instead of bus; unicast instead of broadcast
"interconnect"
distributed directory to track which cores cache each line
unicast find to directory

Q: why do we need locks if we have cache coherence?
cache coherence ensures that cores read fresh data
locks avoid lost updates in read-modify-write cycles
and prevent anyone from seeing partially updated data structures

people build locks from h/w-supported atomic instructions
xv6 uses atomic exchange
other locks use test-and-set, atomic increment, &c
the __sync_... functions in the handout turn into atomic instructions

how does the hardware implement atomic instructions?
get the line in M mode
defer coherence msgs
do all the steps (e.g. read old value, write new value)
resume processing msgs

what is performance of locks?
assume N cores are waiting for the lock
how long does it take to hand off the lock?
from previous holder to next holder

bottleneck is usually the interconnect
so we'll measure cost in terms of # of msgs

what performance could we hope for?
if N cores waiting,
get through them all in O(N) time
so each critical section and handoff takes O(1) time
i.e. does not increase with N

test&set spinlock (xv6/jos)
waiting cores repeatedly execute e.g. atomic exchange
Q: is that a problem?
yes!
we don't care if waiting cores waste their own time
we do care if waiting cores slow lock holder!
time for critical section and release:
holder must wait in line for access to bus
so holder's mem ops take O(N) time
so handoff time takes O(N)

Q: is O(N) handoff time a problem?
yes! we wanted O(1) time
O(N) per handoff means all N cores takes O(N^2) time, not O(N)

ticket locks:
goal: read-only spin loop, rather than repeated atomic instruction
goal: fairness (turns out t-s locks aren't fair)
idea: assign numbers, wake up one at a time
avoid constant t-s atomic instructions by waiters
Q: why is it cheaper than t-s lock?
Q: why is it fair?
time analysis:
what happens in acquire?
atomic increment -- O(1) broadcast msg
just once, not repeated
then read-only spin, no cost until next release
what happens after release?
invalidate msg for now_serving
N "find" msgs for each core to read now_serving
so handoff has cost O(N)
note: it was *reading* that was costly!
oops, just as bad O() cost as test-and-set

jargon: test-and-set and ticket locks are "non-scalable" locks
== cost of single handoff increases with N

is the cost of non-scalable locks a serious problem?
after all, programs do lots of other things than locking
maybe locking cost is tiny compared to other stuff

see paper's Figure 2
let's consider Figure 2(c), PFIND -- parallel find
x-axis is # of cores, y-axis is finds completed per second (total throughput)
why does it go up?

why does it level off?
why does it go *down*?
what governs how far up it goes -- i.e. the max throughput?
why does it go down so steeply?

reason for suddenness of collapse
serial section takes 7% on one core (Figure 3, last column)
so w/ 14 cores you'd expect just one or two in crit section
so it seems odd that collapse happens so soon
BUT:
once P(two cores waiting for lock) is substantial,
critical section + handoff starts taking longer
so starts to be more than 7%
so more cores end up waiting
so N grows, and thus handoff time, and thus N...

some perspective
acquire(l)
x++
release(l)
surely a critical section this short cannot affect overall performance?
takes a few dozen cycles if same core last held the lock (still in M)
everything operates out of the cache, very fast
a hundred if lock not held, some other core previously held
10,000 if contended by dozens of cores
many kernel operations only take a few 100 cycles total
so a contended lock may increase cost not by a few percent
but by 100x!

how to make locks scale well?
we want just O(1) msgs during a release
how to cause only one core to read/write lock after a release?
how to wake up just one core at a time?

test-and-set with exponential backoff (t_s_exp_acquire):
goal: avoid everyone jumping in at once
space out attempts to acquire lock
simultaneous attempts were reason for O(N) release time w/ t-s
if total rate of tries is low, only one core will attempt per release
why not constant delay?
each core re-tries after random delay with constant average
hard to choose delay time
too large: waste
too small: all N cores probe mult times/crit, so O(N) release time
why exponential backoff?
i.e. why start with small delay, double it?
try to get lucky at first (maybe only a few cores attempting)
doubling means takes only a few attempts until delay >= N * crit section
i.e. just one attempt per release
illustration:
eventually will be roughly one probe per critical section time
then all will complete in that backoff round
can we analyze # of probes?
not that easy

suppose takes time O(N) for all cores to succeed
how many probes does each core make in time N? logN
so total probes: N*logN
so cost per release: O(logN)
not O(1), but much better than O(N)
problem: unlikely to be fair!
some cores will have much lower delays than others
will win, and come back, and win again
some cores will have huge delays, will sit idle
doing no harm, but doing no work

anderson:
goal: O(1) release time, and fair
what if each core spins on a *different* cache line?
acquire cost?
atomic increment, then read-only spin
release cost?
invalidate next holder's slots[]
only they have to re-load
no other cores involved
so O(1) per release -- victory!
problem: high space cost
N slots per lock
often much more than size of protected object

MCS
[just diagram, no code]
goal: as scalable as anderson, but less space used
idea: linked list of waiters per lock
idea: one list element per thread, since a thread can wait on only one lock
so total space is O(locks + threads), not anderson't O(locks*threads)
acquire() pushes caller's element at end of list
caller then spins on a variable in its own element
release() wakes up next element, pops its own element
change in API (need to pass qnode to acquire and release to qnode allocation)

performance of scalable locks?
figure 10 shows ticket, MCS, and optimized backoff
cores on x-axis, total throughput on y-axis
benchmark acquires and releases, critical section dirties four cache lines
Q: why doesn't throughput go up as you add more cores?
ticket is best on two cores -- just one atomic instruction
ticket scales badly: cost goes up with more cores
MCS scales well: cost stays the same with more cores

Figure 11 shows uncontended cost
very fast if no contention!
ticket:
acquire uses a single atomic instruction, so 10s more expensive than release
some what more expensive if so emother core had it last

Concl.
use scalable locks
even better: fix the underlying problem!

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6828 2011 L17: Lock-free coordination

Required reading: Linux scalability

Plan
  programming without locks
    example: lists
  rcu: plan widely-used in Linux
  paper discussion

Problem:
  Locks limit scalability (serialize lock holders)
  Transferring lock from one holder to another is expensive
    Even when locks are scalable
   Can we do better?
      lock-free concurrent data structures

Example: a stack
  Sequential program:

```
struct element {
   int key;
   int value;
   struct element *next;
};

struct element *top;

void push(struct element *e) {
   e->next = top;
   top = e;
}

struct element *pop(void) {
   struct element *e = top;
   top = e->next;
   return e;
}

int search(int key) {
   struct element *e = top;
   while (e) {
      if (e->key == key)
         return e->value;
      e = e->next;
   }
   return -1;
}
```

this is clearly not going to work on a concurrent system
    what is a race?

global spinlock: correct, but what about performance?

how many concurrent ops? just one CPU at a time
bus interactions? bounce cache line for both reads, writes

global read-write lock: correct, but what about performance
how many concurrent ops? theoretically, could do one per CPU
bus interactions? bounce cache line for both reads, writes
draw timing diagram of CPU interactions
we might be slower on two CPUs than on a single CPU!

is it going to get better if we allocate a read-write lock for each item?
concurrency still possible but penalty even worse: N_{elem}
cache line bounces for each search traversal

other possible solutions
partition data into n lists (e.g., n free lists, one per core)
if one runs out of memory, steal memory from other core's free lists

why do we want to avoid locks?
performance
complexity
deadlock
priority inversion

what's the plan?
reduce the operation we want to perform to some atomic x86 instruction
x86 LOCK prefix makes many read-modify-write instructions atomic
simple example: implement atomic counters by adding LOCK prefix
most general thing is cmpxchg (which we seen many times now)

```
int cmpxchg(int *addr, int old, int new) {
  int was = *addr;
  if (was == old)
    *addr = new;
  return was;
}
```

cmpxchg can be used to implement locks, but we can also use it
directly for concurrent, correct access to the linked list.

example: concurrent stack with out locks

```
void push(struct element *e) {
again:
  e->next = top;
  if (cmpxchg(&top, e->next, e) != e->next)
    goto again;
}

struct element *pop(void) {
again:
  struct element *e = top;
  if (cmpxchg(&top, e, e->next) != e)
    goto again;
  return e;
```

}

search can be the same as in the non-concurrent case (almost..)

why is this better than not having locks?
readers no longer generate spurious updates, which incurred performance hit
may be not having to think about deadlock is great

problem 1: lock-free data structures require careful design, hw support
suppose we want to remove arbitrary elements, not just the first one
what could go wrong if we try to use the same cmpxchg?
race condition when two processors
one processor deletes a node next to the other node being deleted
need DCAS (double-compare-and-swap) to implement remove properly
must make sure neither previous nor next element changed
x86 hardware doesn't have DCAS
one approach:
on delete, mark node's next pointer to signal node is deleted
now the CAS that removes a subsequent node if this previous node is delete concurrently too

problem 2: memory reuse
when can we free a memory block, if other CPUs could be accessing it?
other CPU's search might be traversing any of the elements

reusing a memory block can corrupt list
stack contains three elements
top -> A -> B -> C
CPU 1 about to pop off the top of the stack,
preempted just before cmpxchg(&top, A, B)
CPU 2 pops off A, B, frees both of them
top -> C
CPU 2 allocates another item (malloc reuses A) and pushes onto stack
top -> A -> C
CPU 1: cmpxchg succeeds, stack now looks like
top -> B -> C
this is called the "ABA problem"
(memory switches from A-state to B-state and back to A-state without being able to tell)

strawman solution for specific problem (actually used by some systems):
type-stable memory (stack-elements never become non-stack-elements)
each stack element has a reuse counter
include generation# along with each pointer (so that cmpxchg notices)

but for more complex data structures this becomes hard to solve
for example, readers must check that data structure hasn't been freed

need a general-purpose garbage-collection plan
see below

RCU: ready-copy update
concurrent data structure approach used in the Linux kernel for read-intensive concurrent data structures
many usages in kernel, for hash tables, lists, etc.
3 components:
- lock-free readers, but serialize writers using locks

- writers don't update in place
  on write, make copy
  update copy
  switch one pointer atomically
  => readers seen atomic switch from old to new
- garbage collection to allow readers to read old versions and avoid ABA problem

Can be applied to many data structures

Example: RCU stack

```
void
rcu_push(int k, int v)
{
  acquire(l);
  push(k, v);
  release(l);
}

elem_t*
rcu_pop()
{
  elem_t *e;
  acquire(l);
  e = pop();
  release(l);
  return e;
}

int
rcu_search(k)
{
  int v;
  v = search(k);
  return v;
}
```

problem: garbage collection
  after pop, can we just free e?
  no! such may be still looking at it!
  in a garbage-collected language, the garbage collection will make this safe

many different gc schemes possible for C
  on free, put element on a list for delayed freeing
  remove an element when sure that no reader is looking at the element

one scheme: epochs
  global epoch counter
  writers increment epoch number
    they hold lock anyway
  on free of element, record epoch number in element
  at beginning of read, threads stores global epoch in thread-local state
  gc: free all elements with epoch # < minimum of all readers

example stack

```
int
rcu_search(k)
{
  int v;
  rcu_start_read()
  v = search(k);
  rcu_end_read()
  return v;
}

rcu_begin_read(int tid)
{
  epochs[tid].epoch = global_epoch;
  __sync_synchronize();
}

void
rcu_end_read(int tid)
{
  epochs[tid].epoch = INF;
}

void free(e)
{
    // record global epoch into e
}

void gc()
{
  unsigned long min = global_epoch;

  for (i = 0; i < nthread; i++) {
    if (min > epochs[i].epoch)
      min = epochs[i].epoch;
  }
  // free all e whose epoch < min
}
```

RCU
- concurrent code very similar to sequential code
- general approach for read-intensive data structures
- but serializes all writers
  - ok for stack case
  - but what if inserting at the beginning and end of list concurrently?
- may make unnecessary copies

Future
- lock-free data structures for readers and writers?
- exist for lists, hash tables, skip lists, etc.

Paper
- What is the current state of scalability in current kernels?
- Many short sections

Partitioning and replication of data structures for scalability
RCU for lock-free sections

Example: exim
Many processes do path name lookup concurrent
Linux has a directory entry cache for mappings from directory entry to inode #
[i#,name] -> file struct
RCU has been used for the directory entry cache
lookups for different pathnames run in parallel without locks
lookups for the same dir entry of a lock

Problem:
Look ups for "/", however, hit the same dentry
Contention on spinlock, and collapse.
Not simple fix:
reads increase refcnt
writes (e.g., rename), too + [i#,name]->file struct

Strawman solution: scalable locks
Avoid collapse, but not more scalability

Solution: lock-free dentry?
add generation #
write: lock dentry, g = gen, gen=0, change, gen=g+1
read:
g <- gen#
if g== 0, use locks (some core is modifying)
copy relevant fields
g1 <- gen#
if g1 != g locking protocol
if match, refcnt++ (if refcnt != 0)
Gen # is well-known trick
Result: all lookups run lock free

Future: ?

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

6.828 2011 Lecture 19: Virtual Machines

Read: A comparison of software and hardware techniques for x86 virtualizaton, Keith Adams and Ole Agesen, ASPLOS 2006.

what's a virtual machine?
  simulation of a computer
  running as an application on a host computer
  accurate
  isolated
  fast

why use a VM?
  one computer, multiple operating systems (OSX and Windows)
  manage big machines (allocate CPUs/memory at o/s granularity)
  kernel development environment (like qemu)
  better fault isolation: contain break-ins

how accurate do we need?
  handle weird quirks of operating system kernels
  reproduce bugs exactly
  handle malicious software
    cannot let guest break out of virtual machine!
  usual goal:
    impossible for guest to distinguish VM from real computer
    impossible for guest to escape its VM
  some VMs compromise, require guest kernel modifications

VMs are an old idea
  1960s: IBM used VMs to share big machines
  1990s: VMWare re-popularized VMs, for x86 hardware

terminology
  [diagram: h/w, VMM, VMs..]
  VMM ("host")
  guest: kernel, user programs
  VMM might run in a host O/S, e.g. OSX
    or VMM might be stand-alone

VMM responsibilities
  divide memory among guests
  time-share CPU among guests
  simulate per-guest virtual disk, network
    really e.g. slice of real disk

why not simulation?
  VMM interpret each guest instruction
  maintain virtual machine state for each guest
    eflags, %cr3, &c
  much too slow!

idea: execute guest instructions on real CPU when possible
  works fine for most instructions

e.g. add %eax, %ebx
how to prevent guest from executing privileged instructions?
could then wreck the VMM, other guests, &c

idea: run each guest kernel at CPL=3
ordinary instructions work fine
privileged instructions will (usually) trap to the VMM
VMM can apply the privileged operation to *virtual* state
not to the real hardware
"trap-and-emulate"

Trap-and-emulate example -- CLI / STI
VMM maintains virtual IF for guest
VMM controls hardware IF
Probably leaves interrupts enabled when guest runs
Even if a guest uses CLI to disable them
VMM looks at virtual IF to decide when to interrupt guest
When guest executes CLI or STI:
Protection violation, since guest at CPL=3
Hardware traps to VMM
VMM looks at *virtual* CPL
If 0, changes *virtual* IF
If not 0, emulates a protection trap to guest kernel
VMM must cause guest to see only virtual IF
and completely hide/protect real IF

trap-and-emulate is hard on an x86
not all privileged instructions trap at CPL=3
popf silently ignores changes to interrupt flag
pushf reveals *real* interrupt flag
all those traps can be slow
VMM must see PTE writes, which don't use privileged instructions

what real x86 state do we have to hide (i.e. != virtual state)?
CPL (low bits of CS) since it is 3, guest expecting 0
gdt descriptors (DPL 3, not 0)
gdtr (pointing to shadow gdt)
idt descriptors (traps go to VMM, not guest kernel)
idtr
pagetable (doesn't map to expected physical addresses)
%cr3 (points to shadow pagetable)
IF in EFLAGS
%cr0 &c

how can VMM give guest kernel illusion of dedicated physical memory?
guest wants to start at PA=0, use all "installed" DRAM
VMM must support many guests, they can't all really use PA=0
VMM must protect one guest's memory from other guests
idea:
claim DRAM size is smaller than real DRAM
ensure paging is enabled
maintain a "shadow" copy of guest's page table
shadow maps VAs to different PAs than guest
real %cr3 refers to shadow page table

virtual %cr3 refers to guest's page table
example:
VMM allocates a guest phys mem 0x1000000 to 0x2000000
VMM gets trap if guest changes %cr3 (since guest kernel at CPL=3)
VMM copies guest's pagetable to "shadow" pagetable
VMM adds 0x1000000 to each PA in shadow table
VMM checks that each PA is < 0x2000000

Why can't VMM just modify the guest's page-table in-place?

also shadow the GDT, IDT
real IDT refers to VMM's trap entry points
VMM can forward to guest kernel if needed
VMM may also fake interrupts from virtual disk
real GDT allows execution of guest kernel by CPL=3

note we rely on h/w trapping to VMM if guest writes %cr3, gdtr, &c
do we also need a trap if guest *read*s?

do all instructions that read/write sensitive state cause traps at CPL=3?
push %cs will show CPL=3, not 0
sgdt reveals real GDTR
pushf pushes real IF
suppose guest turned IF off
VMM will leave real IF on, just postpone interrupts to guest
popf ignores IF if CPL=3, no trap
so VMM won't know if guest kernel wants interrupts
IRET: no ring change so won't restore restore SS/ESP

how can we cope with non-trapping instructions that reveal real state?
modify guest code, change them to INT 3, which traps
keep track of original instruction, emulate in VMM
INT 3 is one byte, so doesn't change code size/layout
this is a simplified version of the paper's Binary Translation

how does rewriter know where instruction boundaries are?
or whether bytes are code or data?
can VMM look at symbol table for function entry points?

idea: scan only as executed, since execution reveals instr boundaries
original start of kernel (making up these instructions):
entry:
pushl %ebp
...
popf
...
jnz x
...
jxx y
x:
...
jxx z
when VMM first loads guest kernel, rewrite from entry to first jump
replace bad instrs (popf) with int3

    replace jump with int3
    then start the guest kernel
  on int3 trap to VMM
    look where the jump could go (now we know the boundaries)
    for each branch, xlate until first jump again
    replace int3 w/ original branch
    re-start
  keep track of what we've rewritten, so we don't do it again

indirect calls/jumps?
  same, but can't replace int3 with the original jump
  since we're not sure address will be the same next time
  so must take a trap every time

ret (function return)?
  == indirect jump via ptr on stack
  can't assume that ret PC on stack is from a call
  so must take a trap every time. slow!

what if guest reads or writes its own code?
  can't let guest see int3
  must re-rewrite any code the guest modifies
  can we use page protections to trap and emulate reads/writes?
    no: can't set up PTE for X but no R
  perhaps make CS != DS
    put rewritten code in CS
    put original code in DS
    write-protect original code pages
  on write trap
    emulate write
    re-rewrite if already rewritten
    tricky: must find first instruction boundary in overwritten code

do we need to rewrite guest user-level code?
  technically yes: SGDT, IF
  but probably not in practice
  user code only does INT, which traps to VMM

how to handle pagetable?
  remember VMM keeps shadow pagetable w/ different PAs in PTEs
  scan the whole pagetable on every %cr3 load?
    to create the shadow page table

what if guest writes %cr3 often, during context switches?
  idea: lazy population of shadow page table
  start w/ empty shadow page table (just VMM mappings)
  so guest will generate many page faults after it loads %cr3
  VMM page fault handler just copies needed PTE to shadow pagetable
    restarts guest, no guest-visible page fault

what if guest frequently switches among a set of page tables?
  as it context-switches among running processes
  probably doesn't modify them, so re-scan (or lazy faults) wasted
  idea: VMM could cache multiple shadow page tables

cache indexed by address of guest pagetable
start with pre-populated page table on guest %cr3 write
would make context switch much faster

what if guest kernel writes a PTE?
store instruction is not privileged, no trap
does VMM need to know about that write?
yes, if VMM is caching multiple page tables
idea: VMM can write-protect guest's PTE pages
trap on PTE write, emulate, also in shadow pagetable

this is the three-way tradeoff the paper talks about
trace costs / hidden page faults / context switch cost
reducing one requires more of the others
and all three are expensive

how to guard guest kernel against writes by guest programs?
both are at CPL=3
delete kernel PTEs on IRET, re-install on INT?

how to handle devices?
trap INB and OUTB
DMA addresses are physical, VMM must translate and check
rarely makes sense for guest to use real device
want to share w/ other guests
each guest gets a part of the disk
each guest looks like a distinct Internet host
each guest gets an X window
VMM might mimic some standard ethernet or disk controller
regardless of actual h/w on host computer
or guest might run special drivers that jump to VMM

Today's paper

Two big issues:
How to cope with instructions that reveal privileged state?
e.g. pushf, looking at low bits of %cs
How to avoid expensive traps?

VMware's answer: binary translation (BT)
Replace offending instructions with code that does the right thing
Code must have access to VMM's virtual state for that guest

Example uses of BT
CLI/STI/pushf/popf -- read/write virtual IF
Detect memory stores that modify PTEs
Write-protect pages, trap the first time, and rewrite
New sequence modifies shadow pagetable as well as real one

How to hide VMM state from guest code?
Since unprivileged BT code now reads/writes VMM state
Put VMM state in very high memory
Use segment limits to prevent guest from using last few pages
But set up %gs to allow BT code to get at those pages

BT challenges
  Hard to find instruction boundaries, instructions vs data
  Translated code is a different size
    Thus code pointers are different
    Program expects to see original fn ptrs, return PCs on stack
    Translated code must map before use
    Thus every RET needs to look up in VMM state

Intel/AMD hardware support for virtual machines
  has made it much easier to implement a VMM w/ reasonable performance
  h/w itself directly maintains per-guest virtual state
    CS (w/ CPL), EFLAGS, idtr, &c
  h/w knows it is in "guest mode"
    instructions directly modify virtual state
    avoids lots of traps to VMM
  h/w basically adds a new priv level
    VMM mode, CPL=0, ..., CPL=3
    guest-mode CPL=0 is not fully privileged
  no traps to VMM on system calls
    h/w handles CPL transition
  what about memory, pagetables?
    h/w supports *two* page tables
    guest page table
    VMM's page table
    guest memory refs go through double lookup
      each phys addr in guest pagetable translated through VMM's pagetable
    thus guest can directly modify its page table w/o VMM having to shadow it
      no need for VMM to write-protect guest pagetables
      no need for VMM to track %cr3 changes
    and VMM can ensure guest uses only its own memory
      only map guest's memory in VMM page table

MIT OpenCourseWare
http://ocw.mit.edu

6.828 Operating System Engineering

Fall 2012

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.